COMP0222 Simultaneous Localisation and Mapping

Lecture 08: Factor Graph SLAM

Simon Julier

s.julier@ucl.ac.uk

Sajad Saeedi

s.saeedi@ucl.ac.uk

Graphical Models Redux

- We have seen we can set up estimation problems in graphs
- The graphs provide an exact description of the probability distribution
- Maximum likelihood estimation is a way to extract estimates without integration
- So…. How do we use it in SLAM?

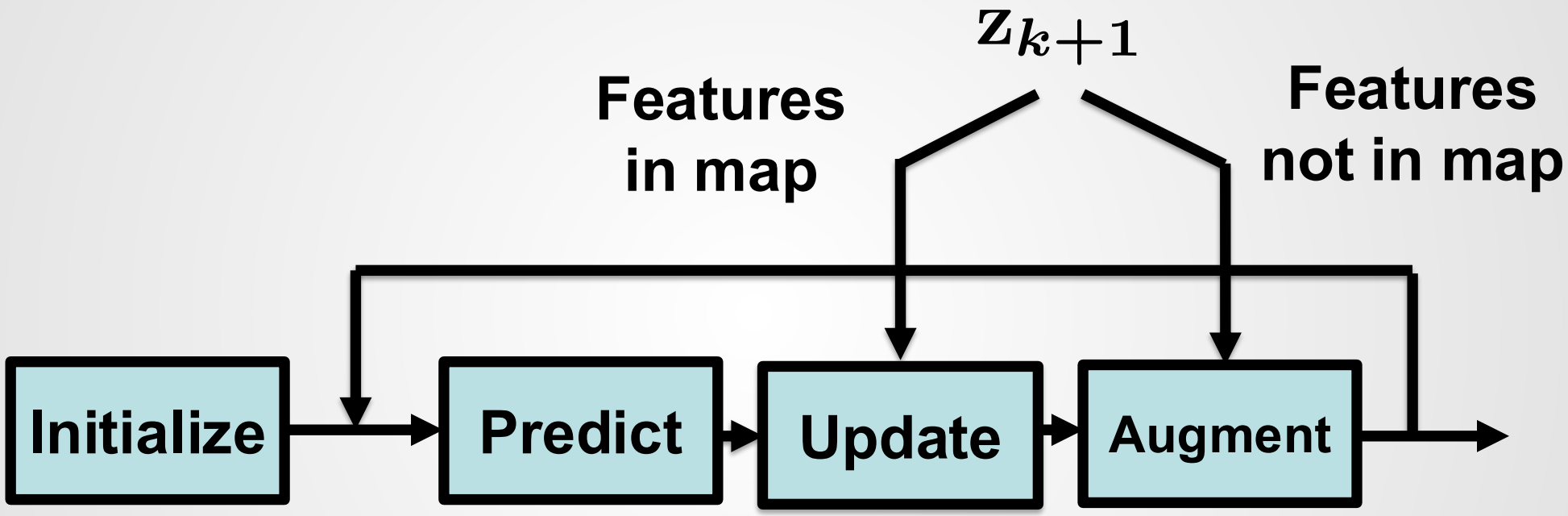
Structure

- Formulating SLAM as a Graph Problem
- SLAM Examples
- Computational and Storage Costs

Formulating SLAM as a Graph Problem

- **Formulating SLAM as a Graph Problem**
- ***SLAM Examples***
- ***Computational and Storage Costs***

SLAM Structure



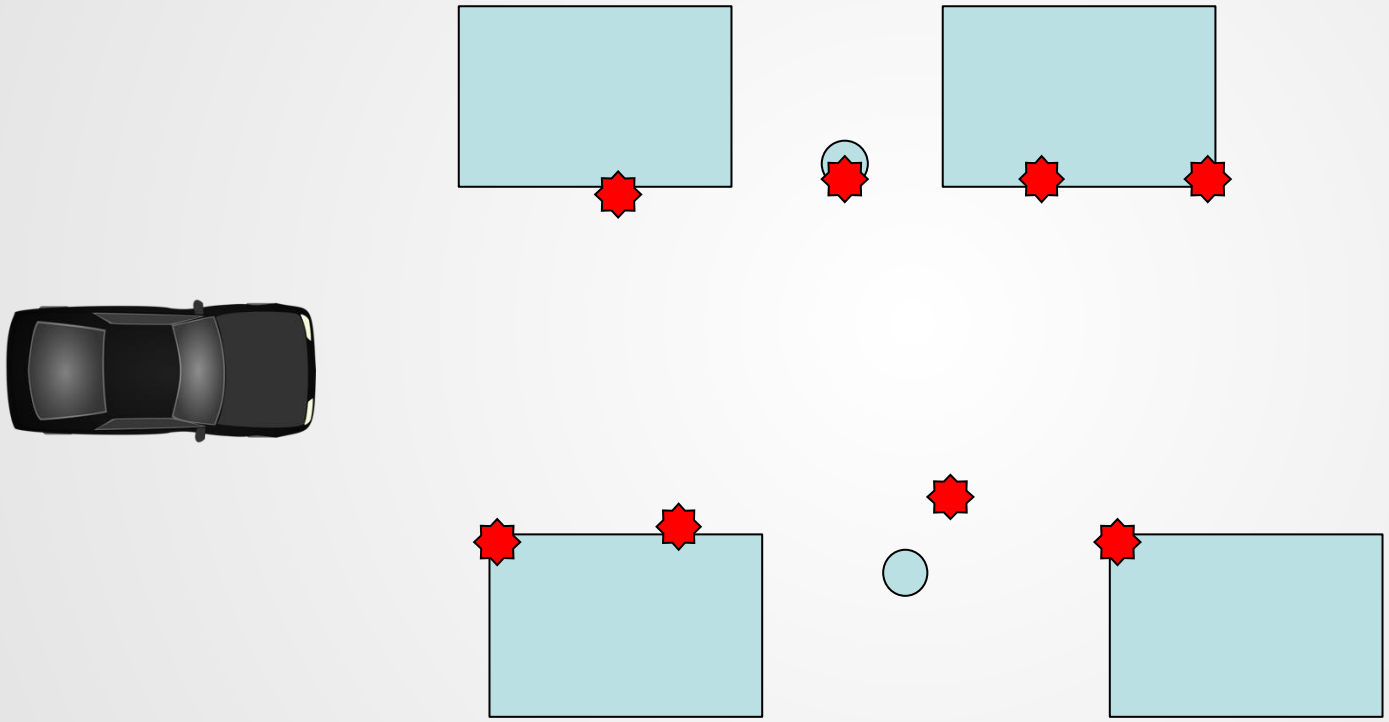
How Do We Use Graphical Models for SLAM?

- Factor Graph based SLAM broadly follows the structure of Kalman filter-based SLAM
- However, the implementation is quite a bit easier – the graph internally manages all the stuff like the cross correlations we had to explicitly work out before
- We'll follow the same pattern we used with KF-SLAM

Factor Graph SLAM: The First Few Timesteps

Timestep	Action
0	Initialize
1	Predict and initalize landmarks 1-4
2	Predict and update landmarks 4, initialize landmarks 5-8

Step 0: Initialize



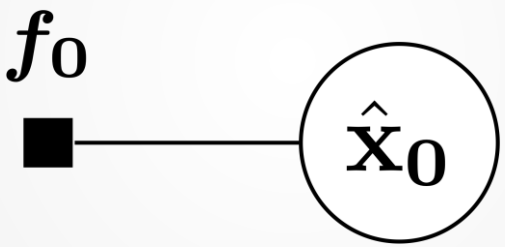
Step 0: Initialization

- The platform starts with an empty map
- Therefore, the only thing we have in the graph is a vertex which corresponds to the state of the platform at time step 0:

$$\hat{\mathbf{x}}_0$$

Step 0: Initialization Graph Structure

- **We can introduce the prior as a factor or a unary edge**



Step 0: Initialization Factor

- **Assuming a Gaussian prior,**

$$f\left(\hat{\mathbf{x}}_0\right) = \mathcal{G}\left(\hat{\mathbf{x}}_0; \bar{\mathbf{x}}_0, \mathbf{P}_0\right)$$

- **The factor is**

$$f\left(\mathbf{x}_0\right) \propto \exp\left\{-\frac{1}{2}\left(\mathbf{x}_0 \boxminus \bar{\mathbf{x}}_0\right)^\top \mathbf{P}_0^{-1}\left(\mathbf{x}_0 \boxminus \bar{\mathbf{x}}_0\right)\right\}$$

Step 0: Initializing the State

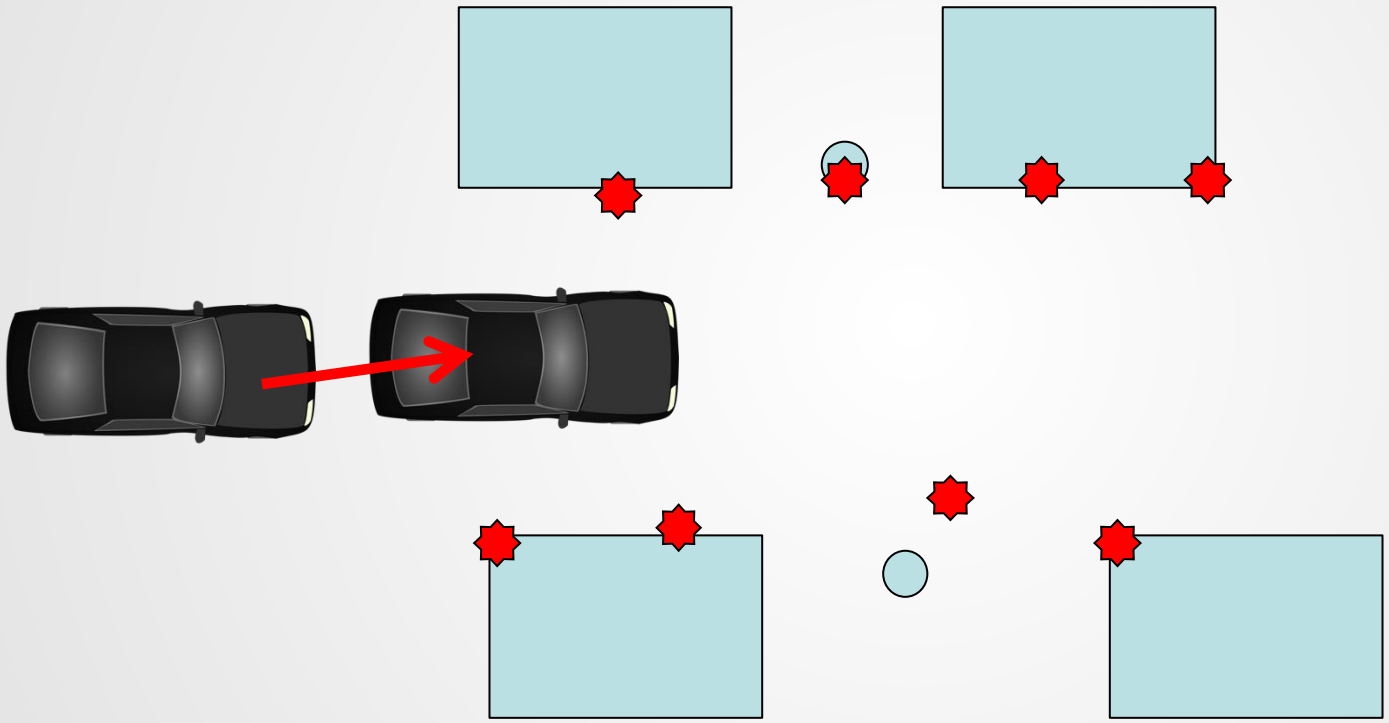
- We have to assign a value to the initial state
- We do this by setting it equal to the mean from the initial condition

$$\hat{\mathbf{x}}_0 = \bar{\mathbf{x}}_0$$

Factor Graph SLAM: The First Few Timesteps

Timestep	Action
~~0~~	~~Initialize~~
1	Predict and initalize landmarks 1-4
2	Predict and update landmarks 4, initialize landmarks 5-8

Step 1: Predict

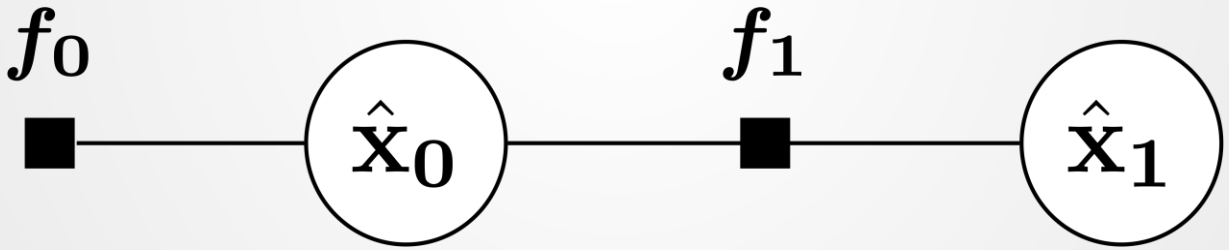


Step 1: Prediction

- We now want to estimate the platform position at time step 1
- Recall that with the graph want to *jointly* estimate the state of the first two steps
- Therefore, the graph needs to contain two platform vertices for the two timesteps

Step 1: Prediction Graph Structure

- We follow the same procedure as the tracking example - create a new vertex for the state at time step 1, and add a (binary) prediction edge



Step 1: Prediction Factor

- Recall the process model is

$$\mathbf{x}_{k+1} = \mathbf{f}\left[\mathbf{x}_k, \mathbf{u}_{k+1}, \mathbf{v}_{k+1}\right]$$

- The factor for it is

$$f\left(\mathbf{x}_{k+1} \middle| \mathbf{x}_k; \mathbf{u}_{k+1}\right) = f\left(\mathbf{v}_{k+1} = \mathbf{e}\left[\mathbf{x}_k, \mathbf{u}_{k+1}, \mathbf{x}_{k+1}\right]\right)$$

Step 1: Prediction Factor

- **Therefore, the factor has the form**

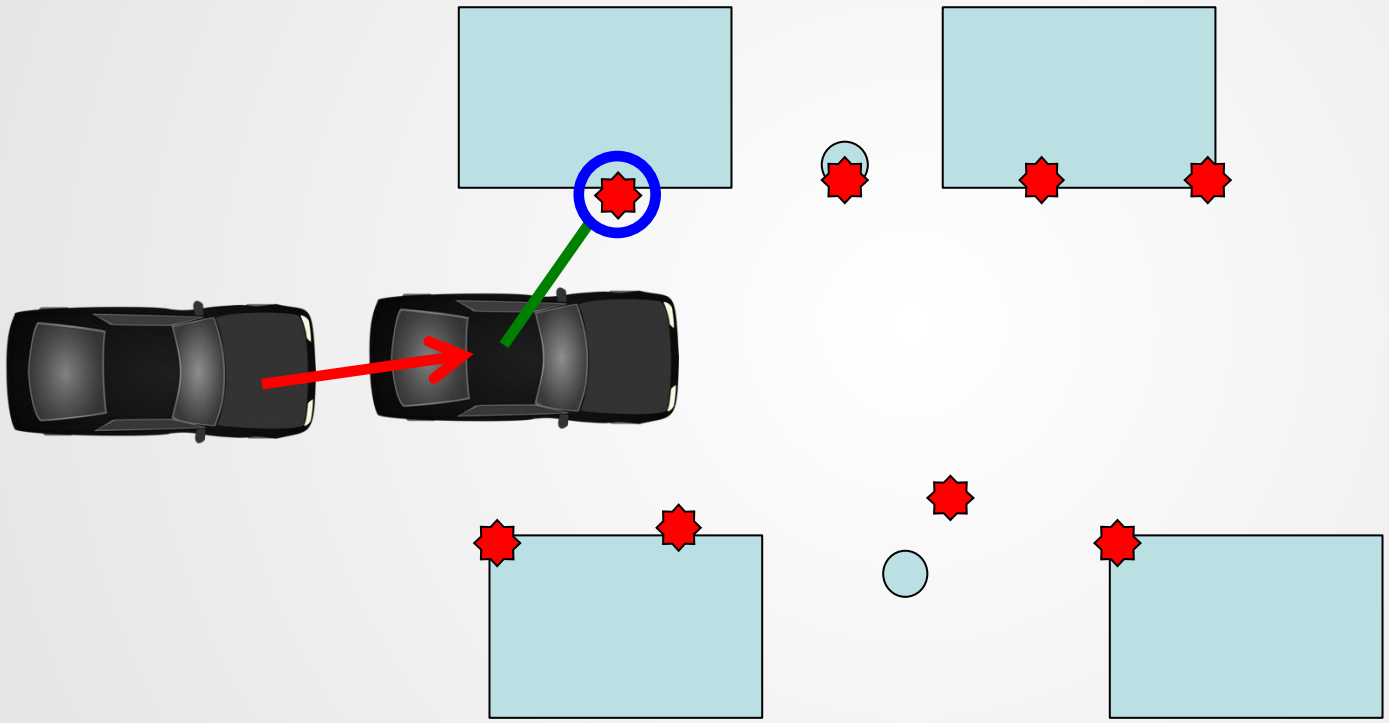
$$f\left(\hat{\mathbf{x}}_{k+1} \middle| \hat{\mathbf{x}}_k; \mathbf{u}_k\right) \propto$$
$$\exp\left\{-\frac{1}{2}\mathbf{e}\left[\hat{\mathbf{x}}_k, \mathbf{u}_{k+1}, \hat{\mathbf{x}}_{k+1}\right]^\top \mathbf{Q}_{k+1}^{-1}\mathbf{e}\left[\hat{\mathbf{x}}_k, \mathbf{u}_{k+1}, \hat{\mathbf{x}}_{k+1}\right]\right\}$$

Step 1: Initializing the New Platform Vertex State

- We could use any logic to assign an initial value
- However, recall that optimizers have a “basin of convergence” so we’d like to initialize as close to true as possible
- Therefore, a common choice is to use the EKF linearization approximation for the mean,

$$\hat{\mathbf{x}}_{k+1} = \mathbf{f}\left[\hat{\mathbf{x}}_k, \mathbf{u}_{k+1}, 0\right]$$

Step 1: Observing Landmarks

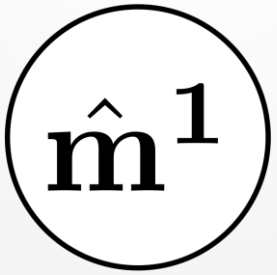


Step 1: Initializing Landmarks

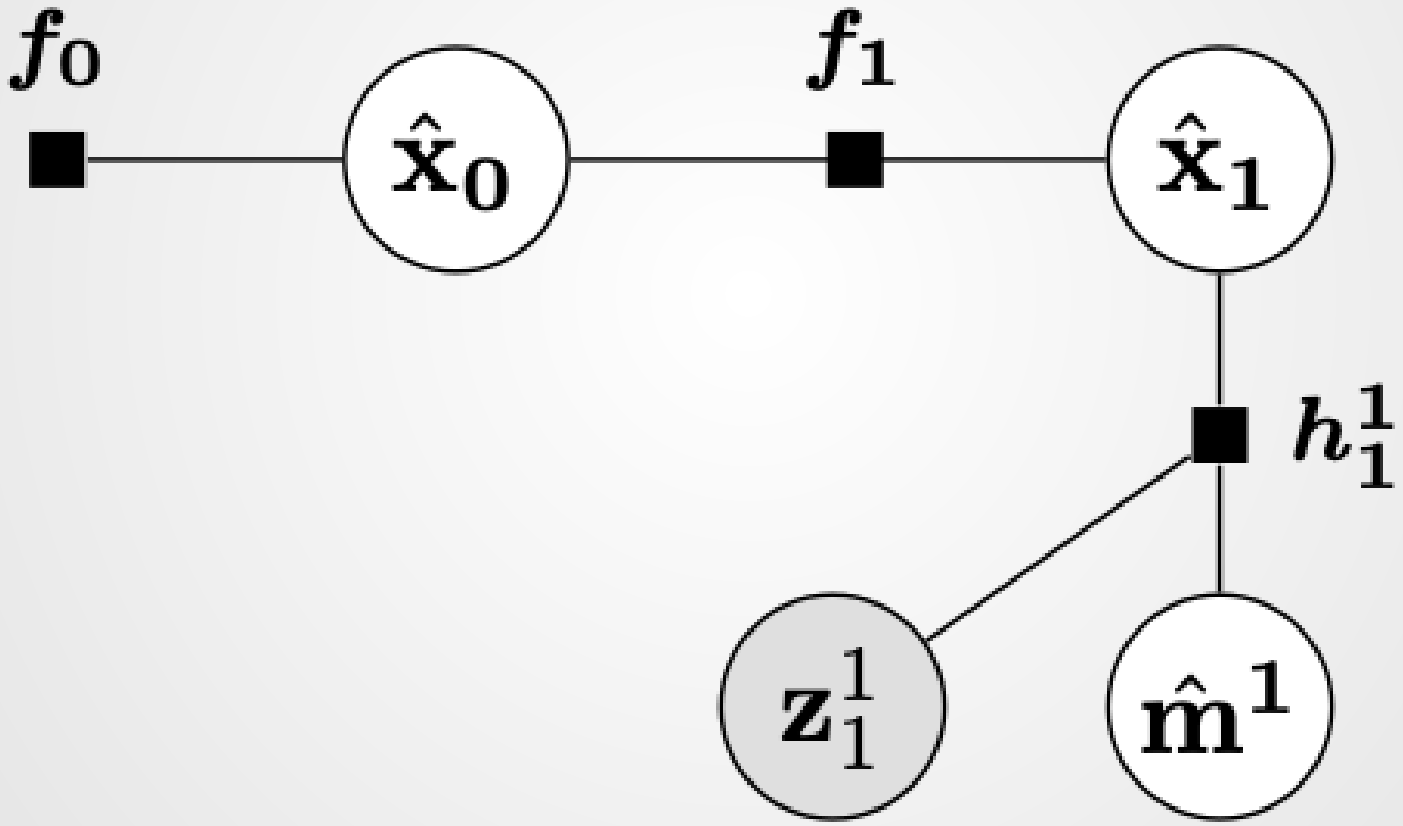
- The SLAM system now observes landmarks 1-4
- We will look at how to initialize one landmark – the procedure simply repeats for the other landmarks

Step 1: Initializing Landmarks

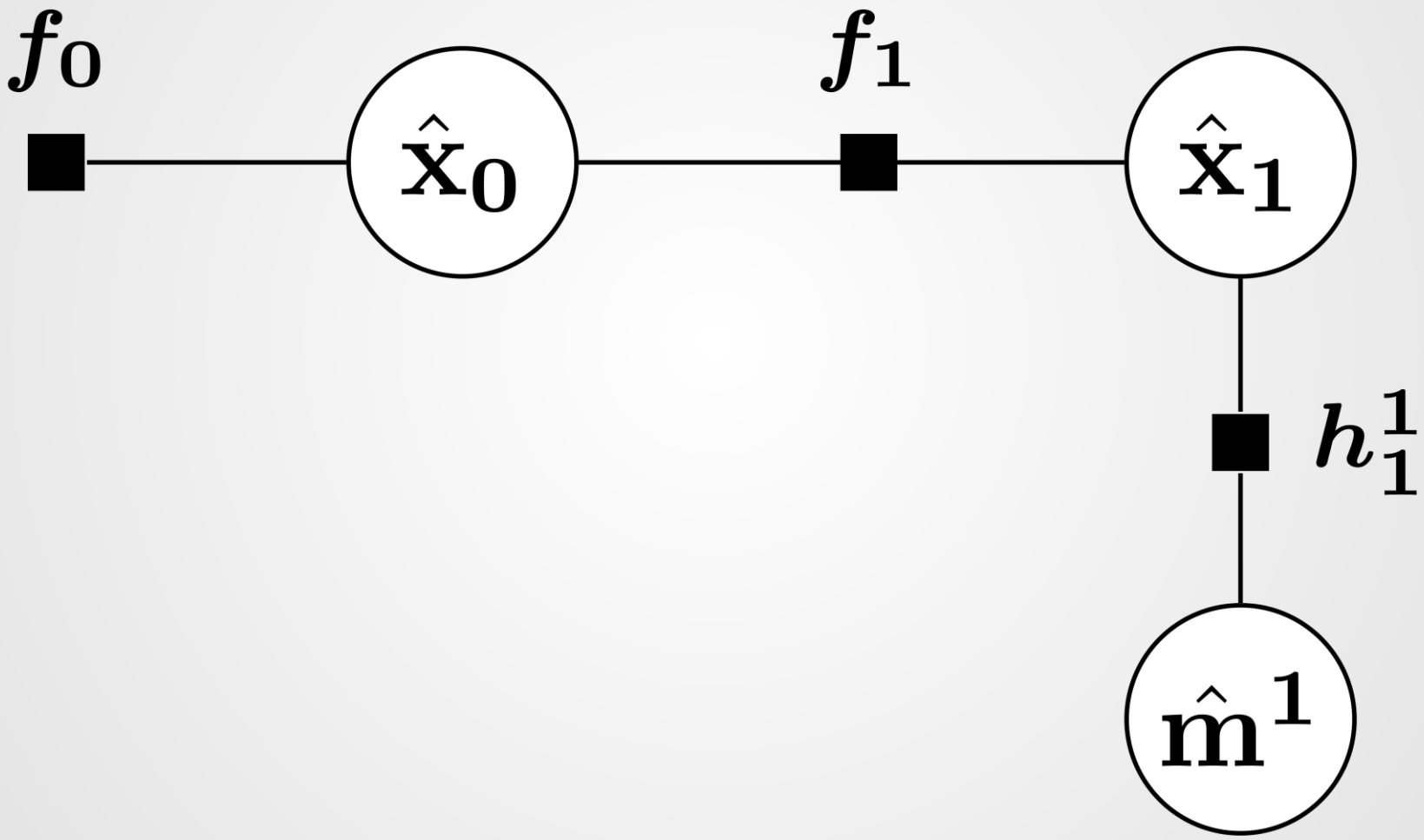
- The first thing is that we need to add a new vertex to the graph which corresponds to the landmark
- Since the landmark is constant over time, we only introduce a *single* vertex and use it for all time



Step 1: Graph Structure



Step 1: Simplified Graph Structure



Step 1: Landmark Initialization

- We now need to choose a factor which links the landmark observation with the platform state
- With the Kalman filter, we had to introduce the augmentation step to do it
- However, in the graph *augmentation is automatic* – every time you add a vertex with an edge to the graph, the back end works out all the correlations, etc. automatically

Step 1: Landmark Initialization Factor

- **Therefore, when initializing a landmark it turns out:**
  - **The factor is *the same type* that you use for a regular observation**
  - **The *only difference* is that you provide an initial value for the landmark estimate**

Step 1: Landmark Factor

- Recall the observation model which links the platform with the landmark state has the form

$$\mathbf{z}_{k+1}^{j} = \mathbf{h}\left[\mathbf{x}_{k+1}, \mathbf{m}^{i_j}, \mathbf{w}_{k+1}^{j}\right]$$

- Therefore, we can rearrange this to give

$$\mathbf{w}_{k+1}^{j} = \mathbf{l}\left[\mathbf{z}_{k+1}^{j}, \mathbf{x}_{k+1}, \mathbf{m}^{i_j}\right]$$

Step 1: Landmark Factor

- **Therefore, the observation likelihood factor has the form**

$$L\left(\hat{\mathbf{x}}_{k+1}, \hat{\mathbf{m}}^{i_j}; \mathbf{z}_{k+1}^{j}\right) \propto$$

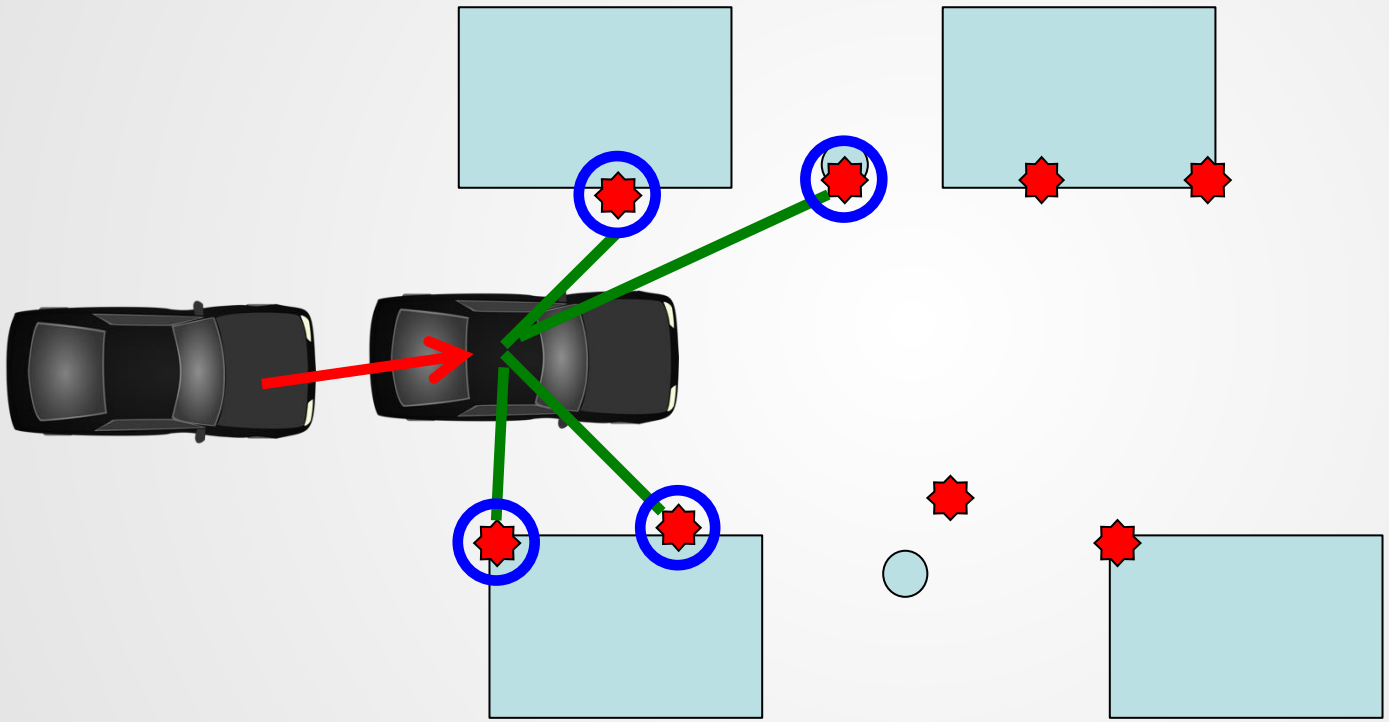
$$\exp\left\{\mathbf{l}\left[\mathbf{z}_{k+1}^{j}, \hat{\mathbf{x}}_{k+1}, \hat{\mathbf{m}}^{i_j}\right]^{\top} \mathbf{R}_{k+1}^{-1} \mathbf{l}\left[\mathbf{z}_{k+1}^{j}, \hat{\mathbf{x}}_{k+1}, \hat{\mathbf{m}}^{i_j}\right]\right\}$$

Step 1: Initializing the New Platform Vertex State

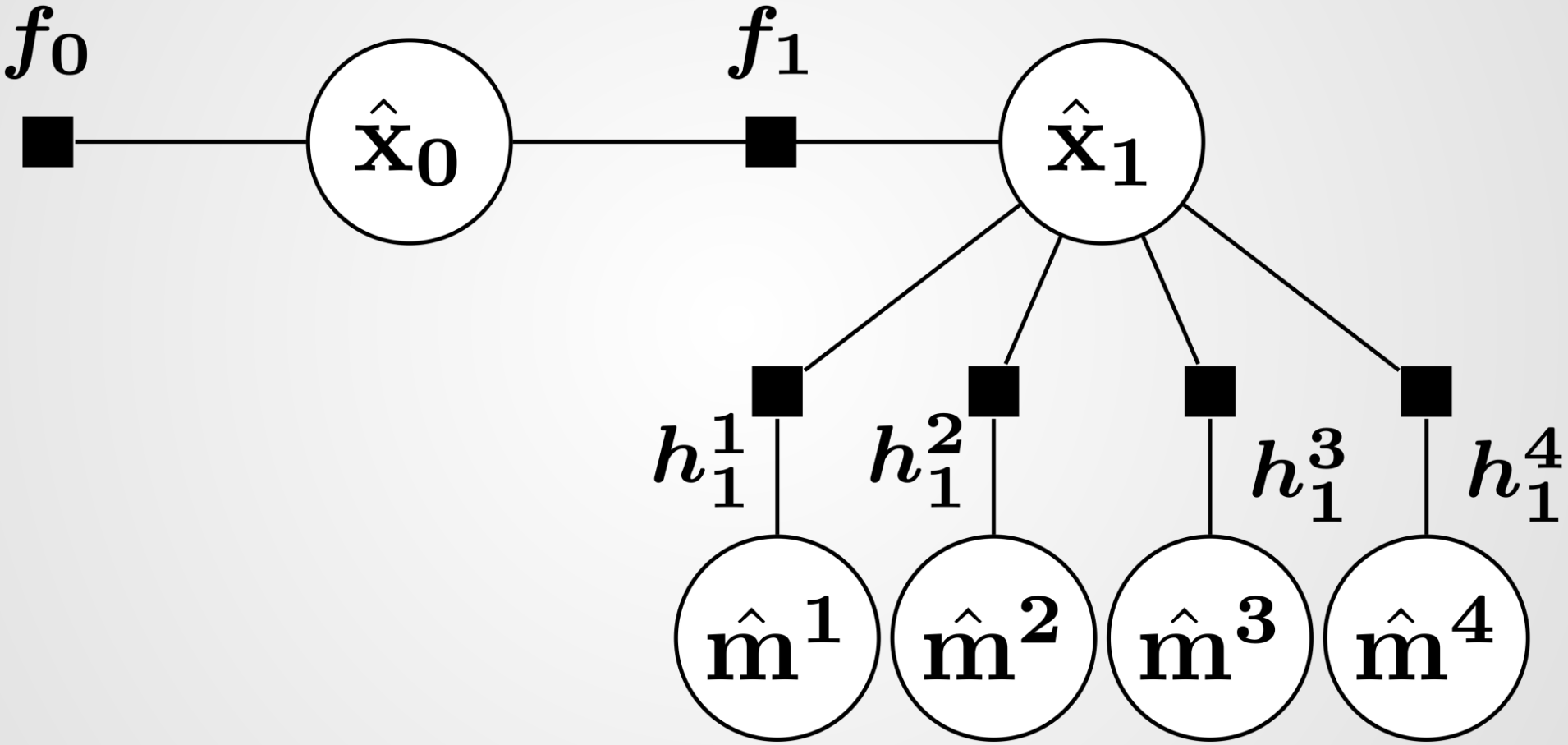
- We still need to initialize the landmark estimate
- As before, we could use any way to do this
- However, we want to try to initialize close to the true condition
- Therefore, we use the inverse observation model with the linearization assumption,

$$\hat{\mathrm{m}}^{i_j} = \mathrm{g}\left[\hat{\mathrm{x}}_{k+1}, \mathrm{z}^{j}_{k+1}, 0\right]$$

Step 1: Rest of the SLAM Observation



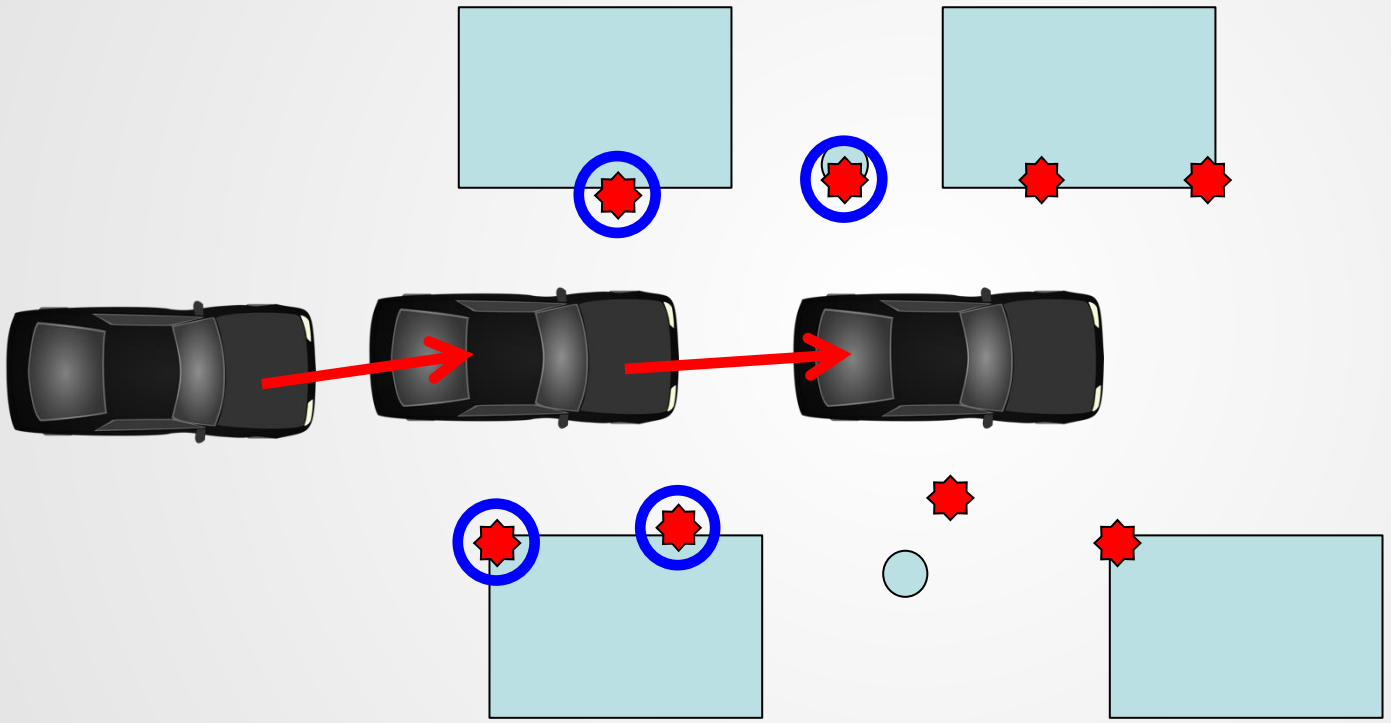
Step 1: Rest of the SLAM Observation



Factor Graph SLAM: The First Few Timesteps

Timestep	Action
~~0~~	~~Initialize~~
~~1~~	~~Predict and initalize landmarks 1-4~~
2	Predict and update landmarks 4, initialize landmarks 5-8

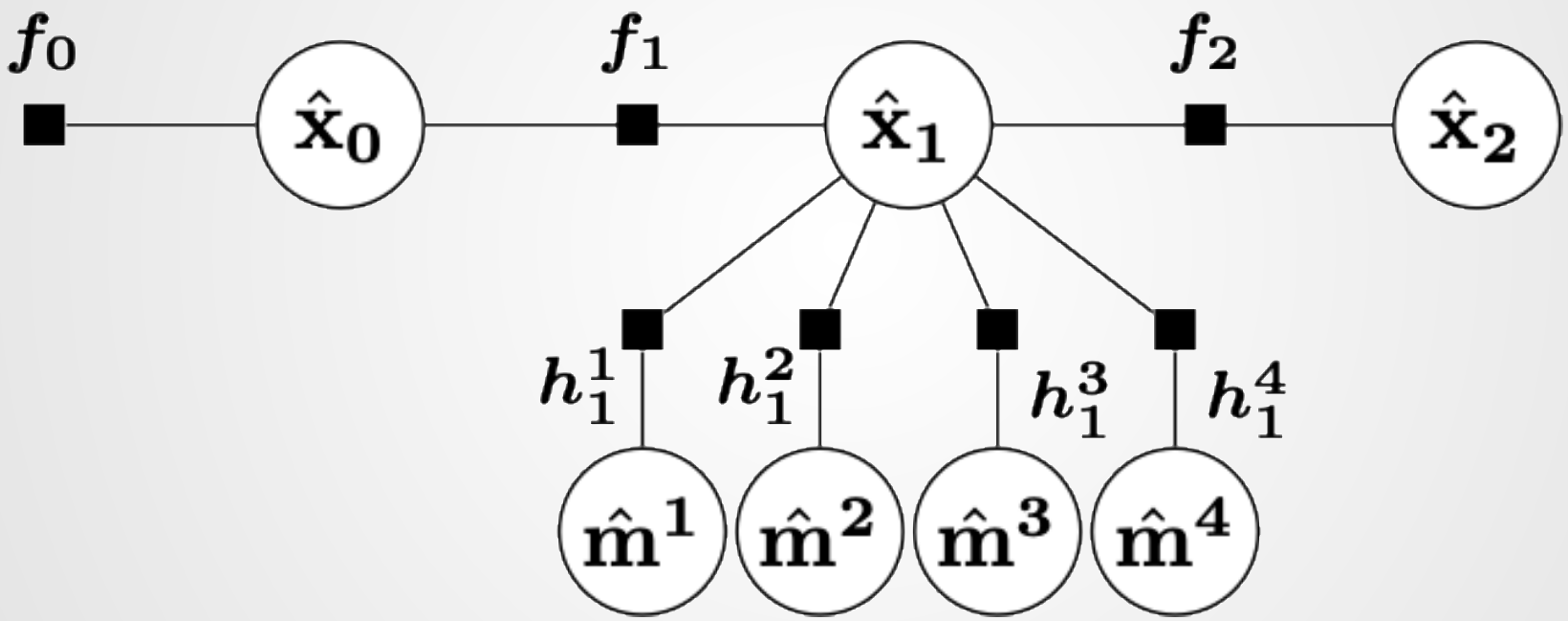
Step 2: Prediction



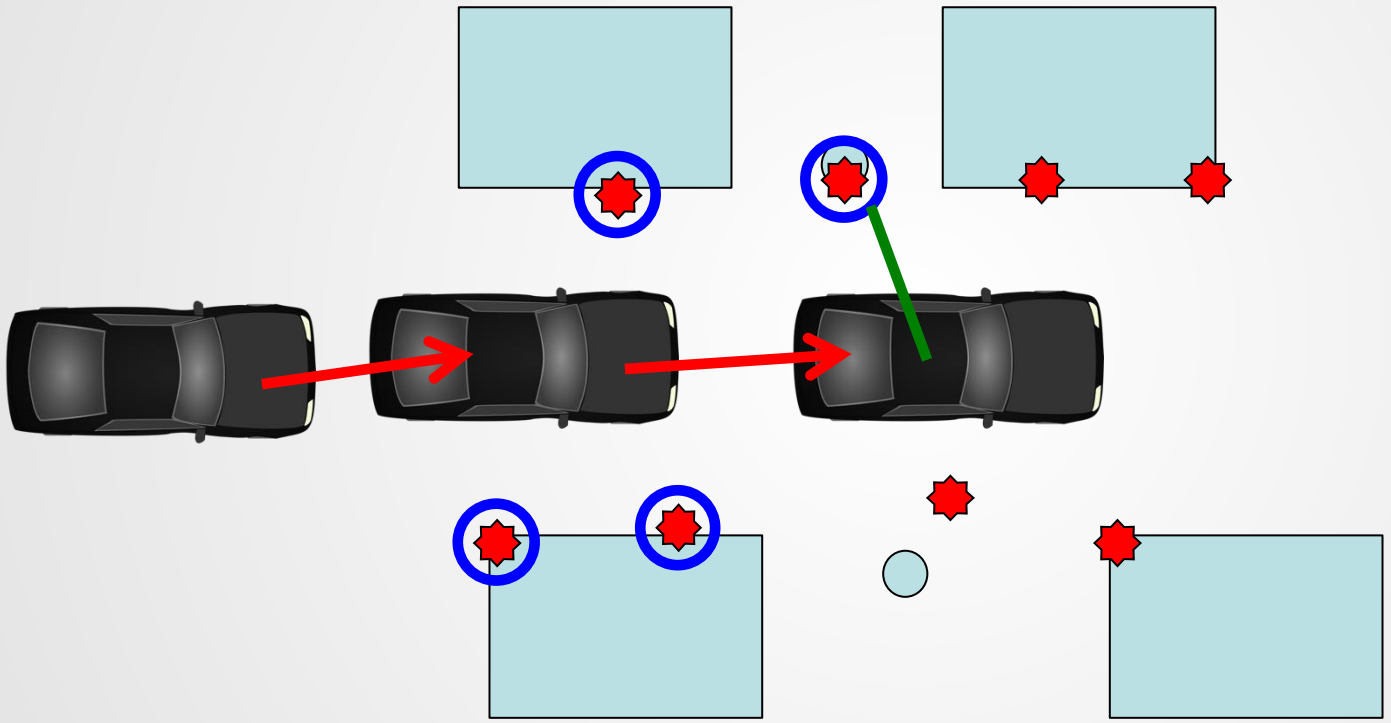
Step 2: Prediction

- We now want to jointly estimate the first three platform poses together with all the landmark estimates
- We do this by augmenting the graph with a new platform state and initializing its position just like we did before

Step 2: Prediction [REDRAW]



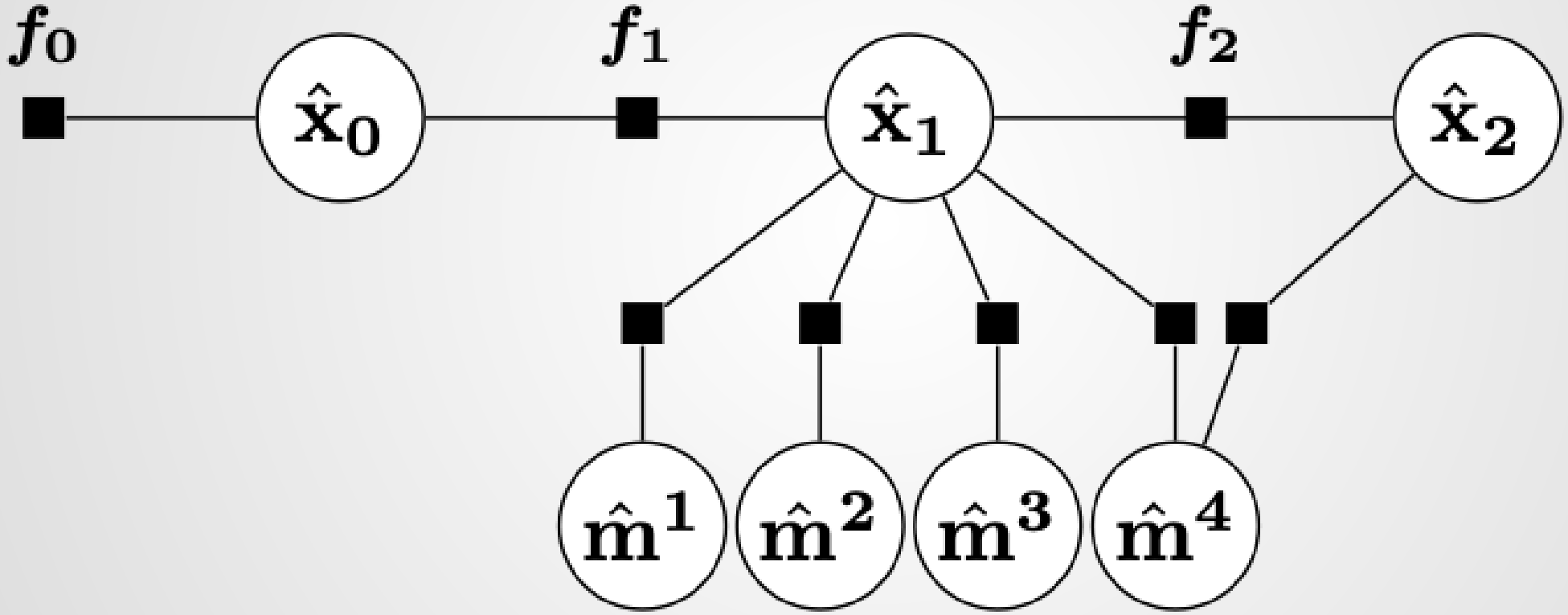
Step 2: Update



Step 2: Update

- We see one landmark ($\mathbf{m}^4$) a second time
- We model this by adding a *second edge* to $\mathbf{m}^4$
- This has the same form as the landmark observation edge we saw with initialization
- However, this time we do not set the estimate in the landmark vertex: the optimizer will update it

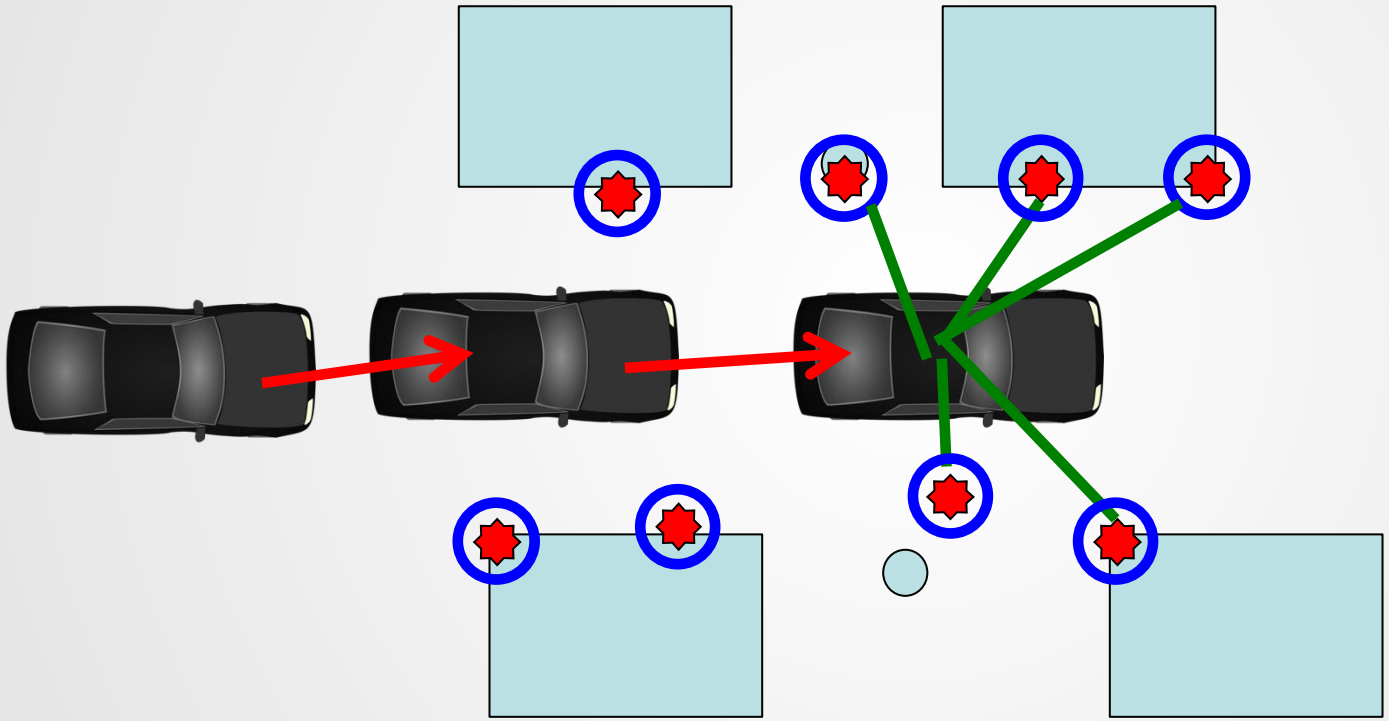
Step 2: Update



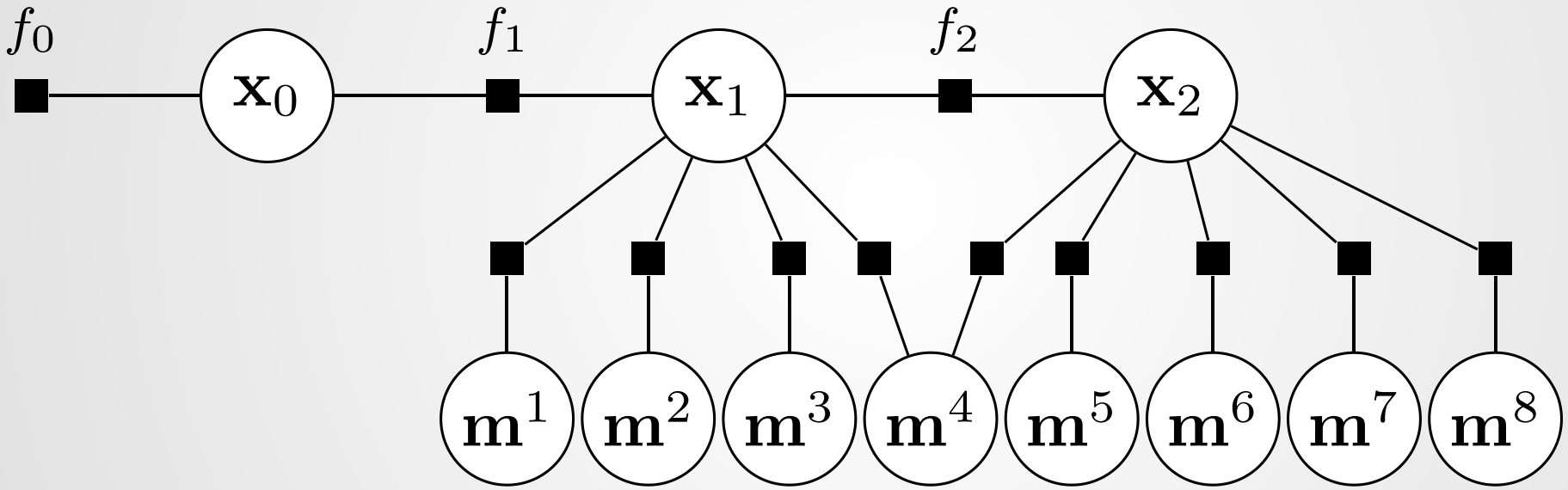
Step 2: Update

- We now need to initialize a new set of landmarks in the map
- We do the same process as before:
  - Create new vertices for the new landmarks
  - Set up the measurement edges
  - Initialize the landmark locations from the inverse observation model

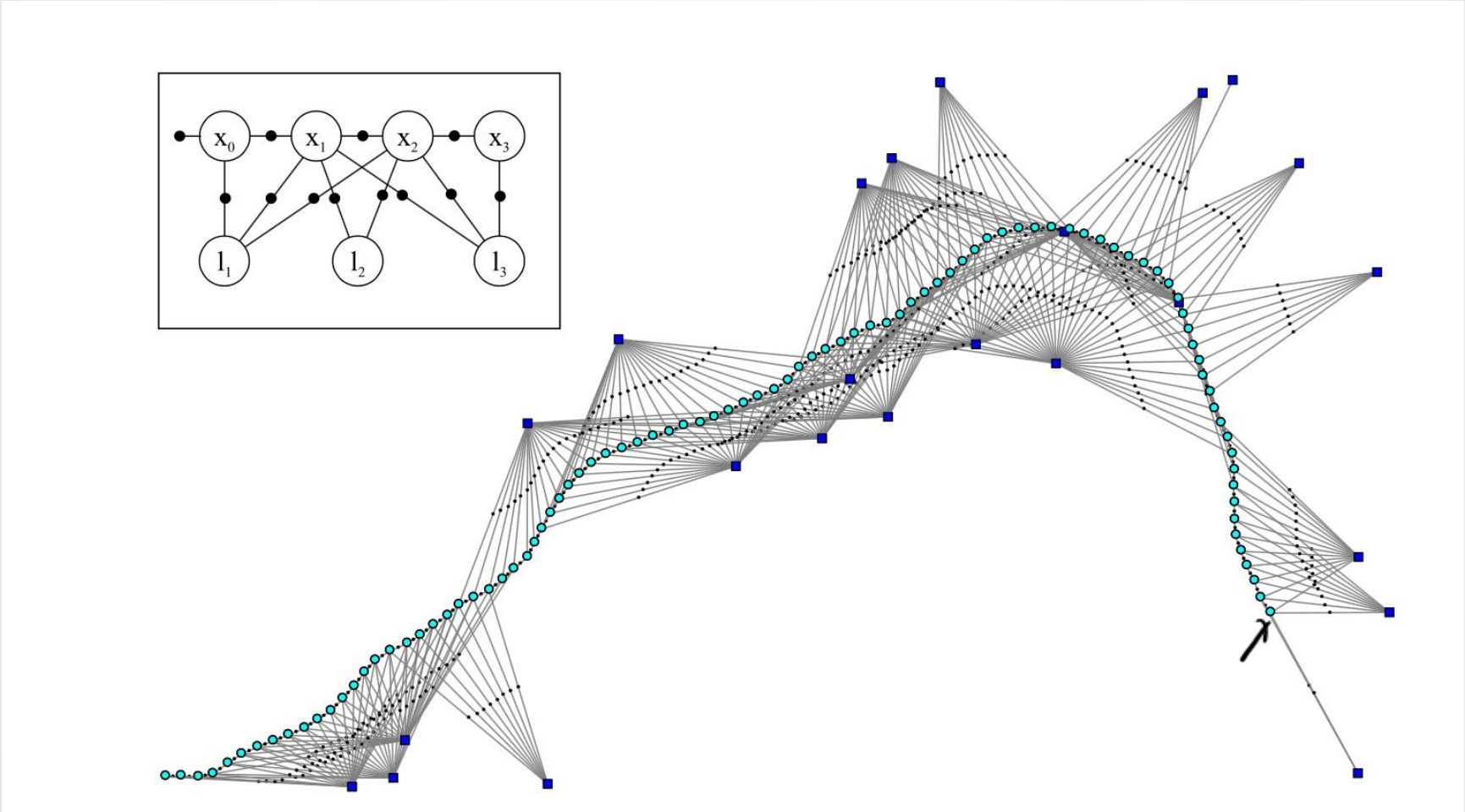
Step 2: Update



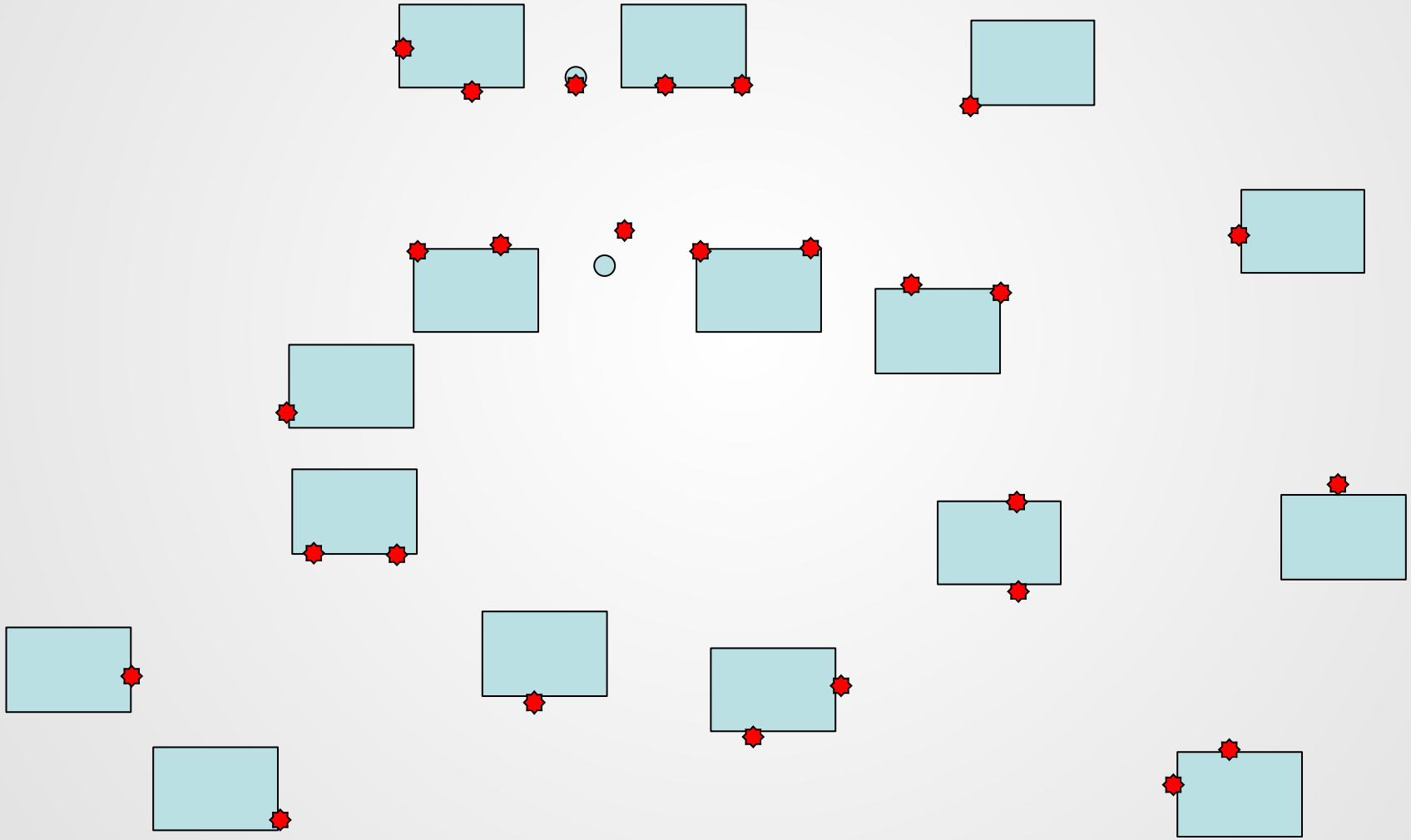
Step 2: Initialize Extra Landmarks



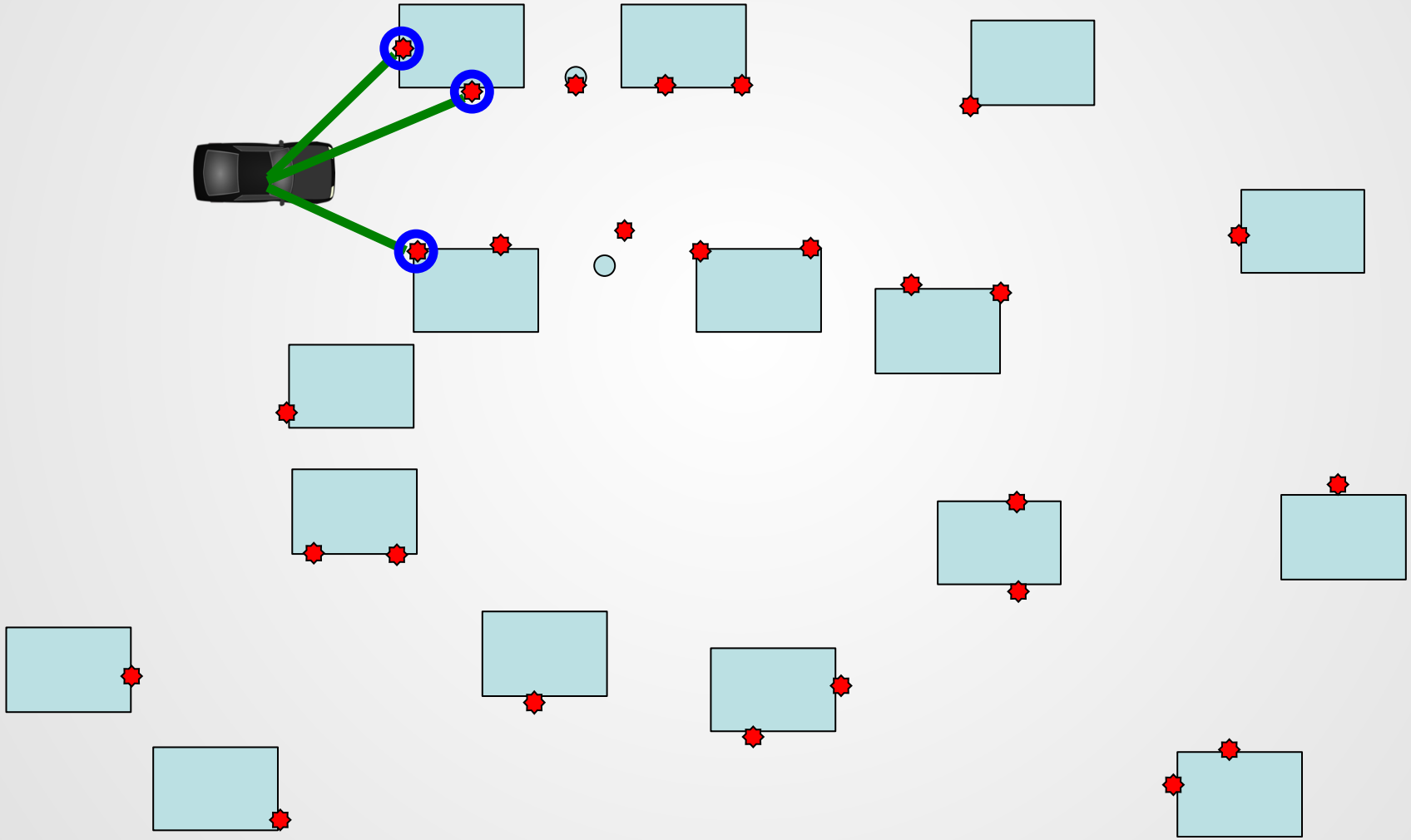
Bayes Net of Actual SLAM Example



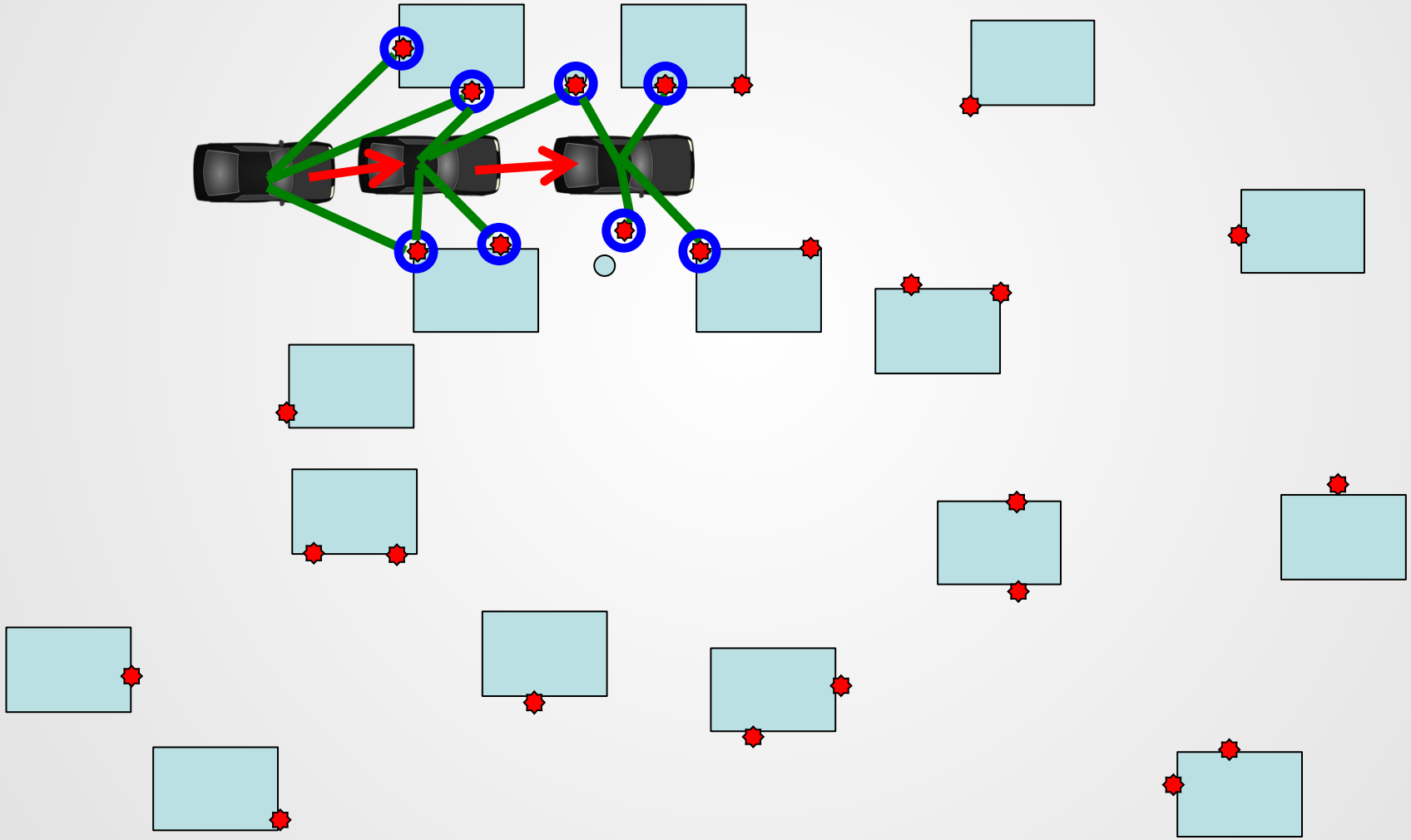
Illustrating Loop Closure



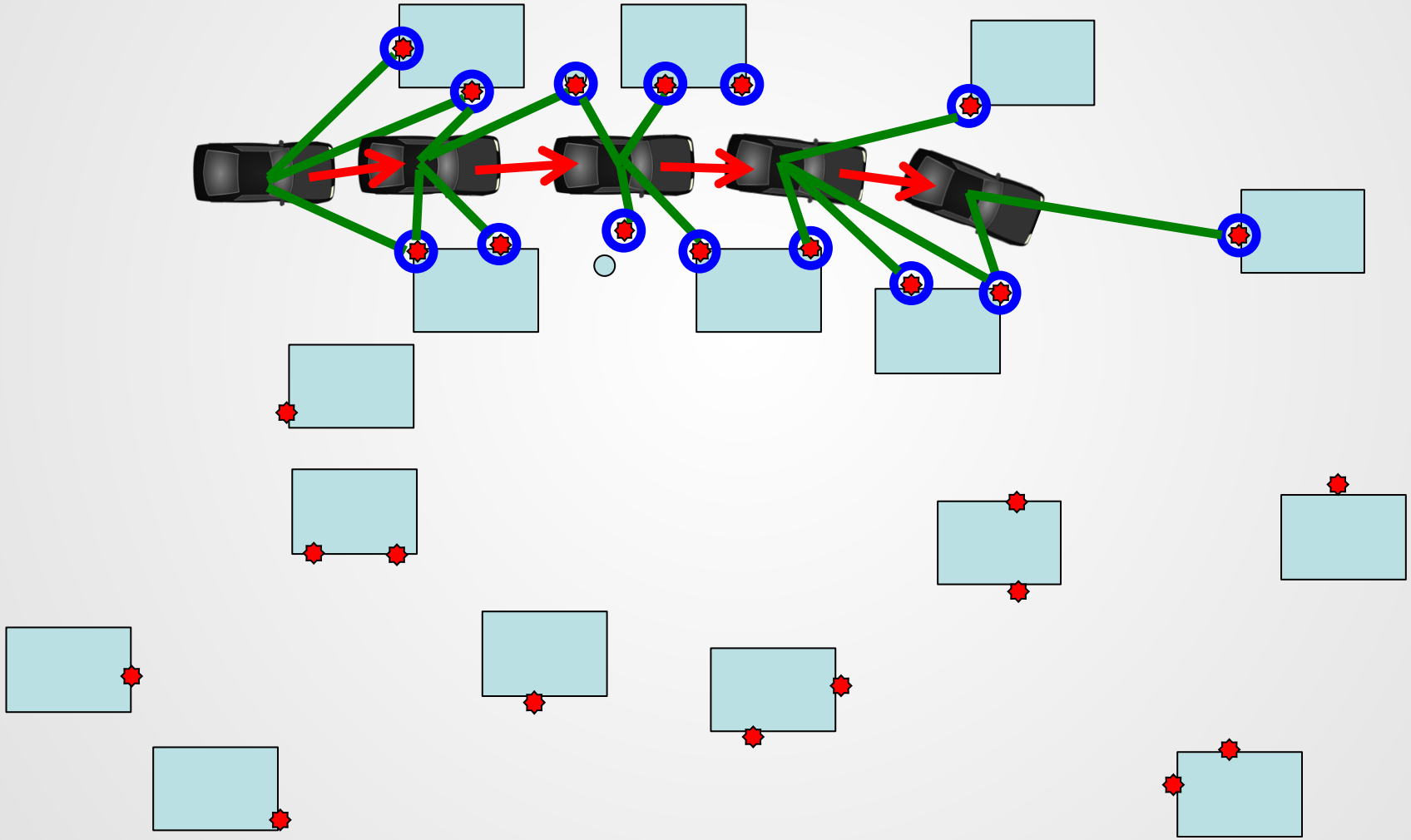
Illustrating Loop Closure



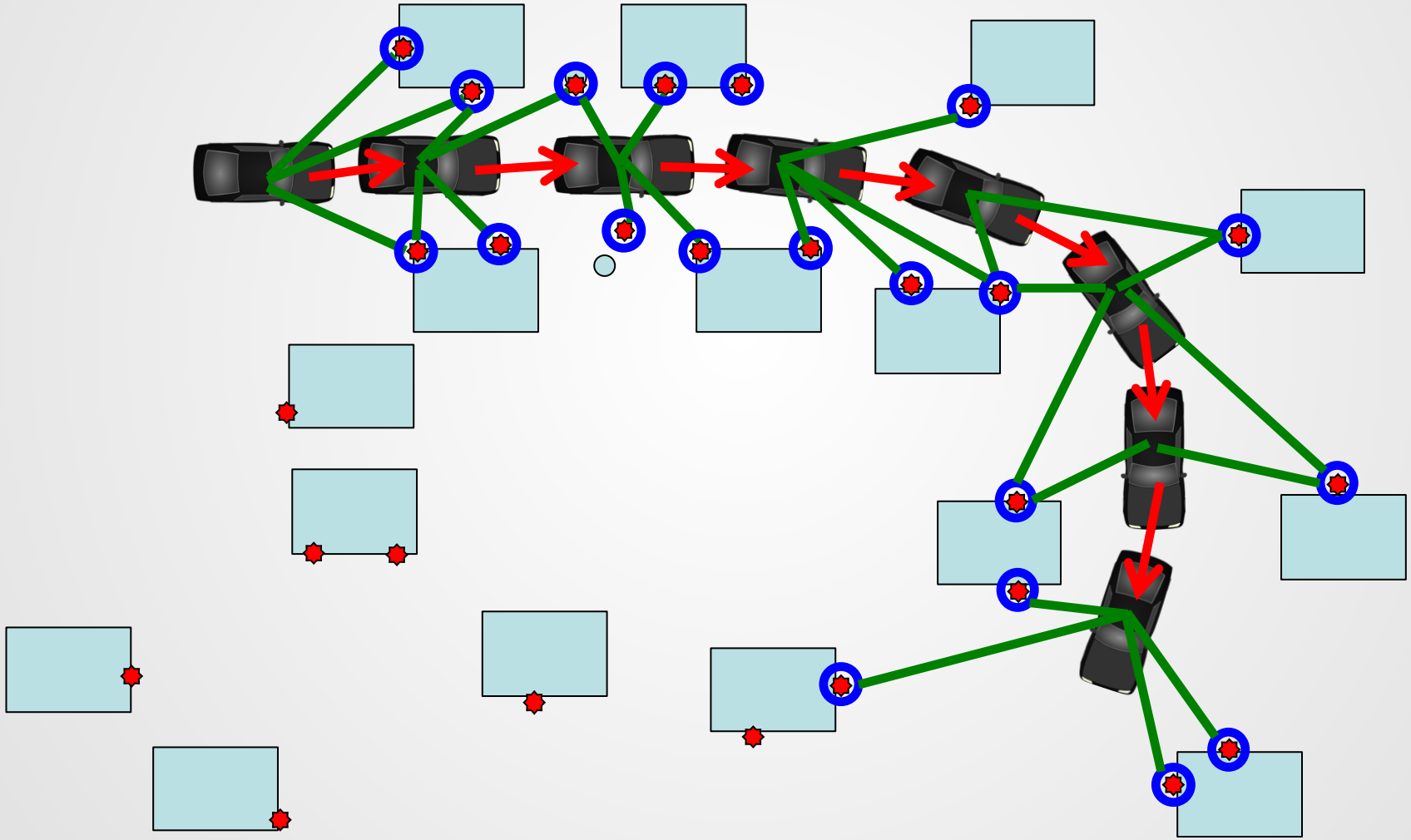
Illustrating Loop Closure



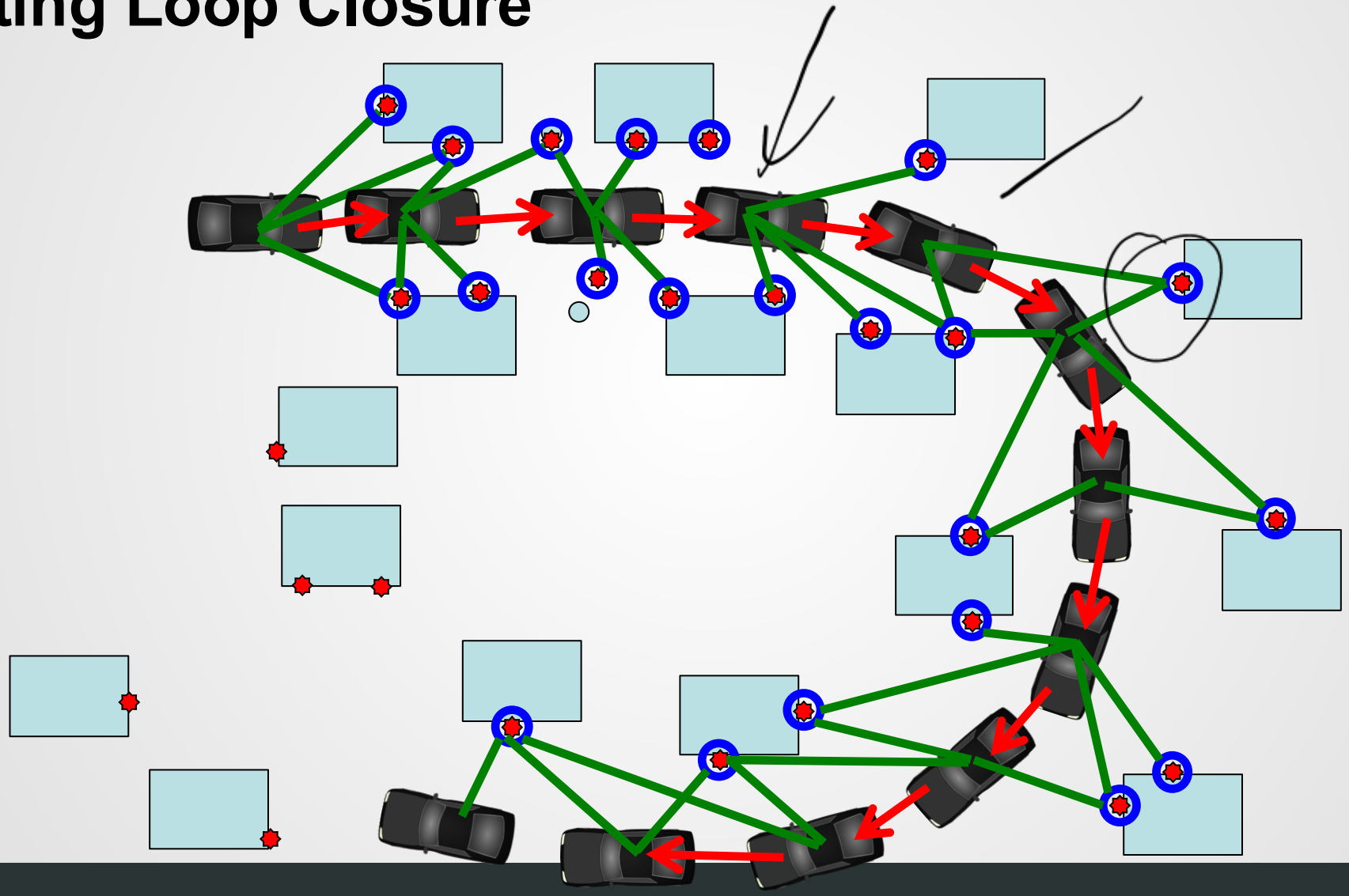
Illustrating Loop Closure



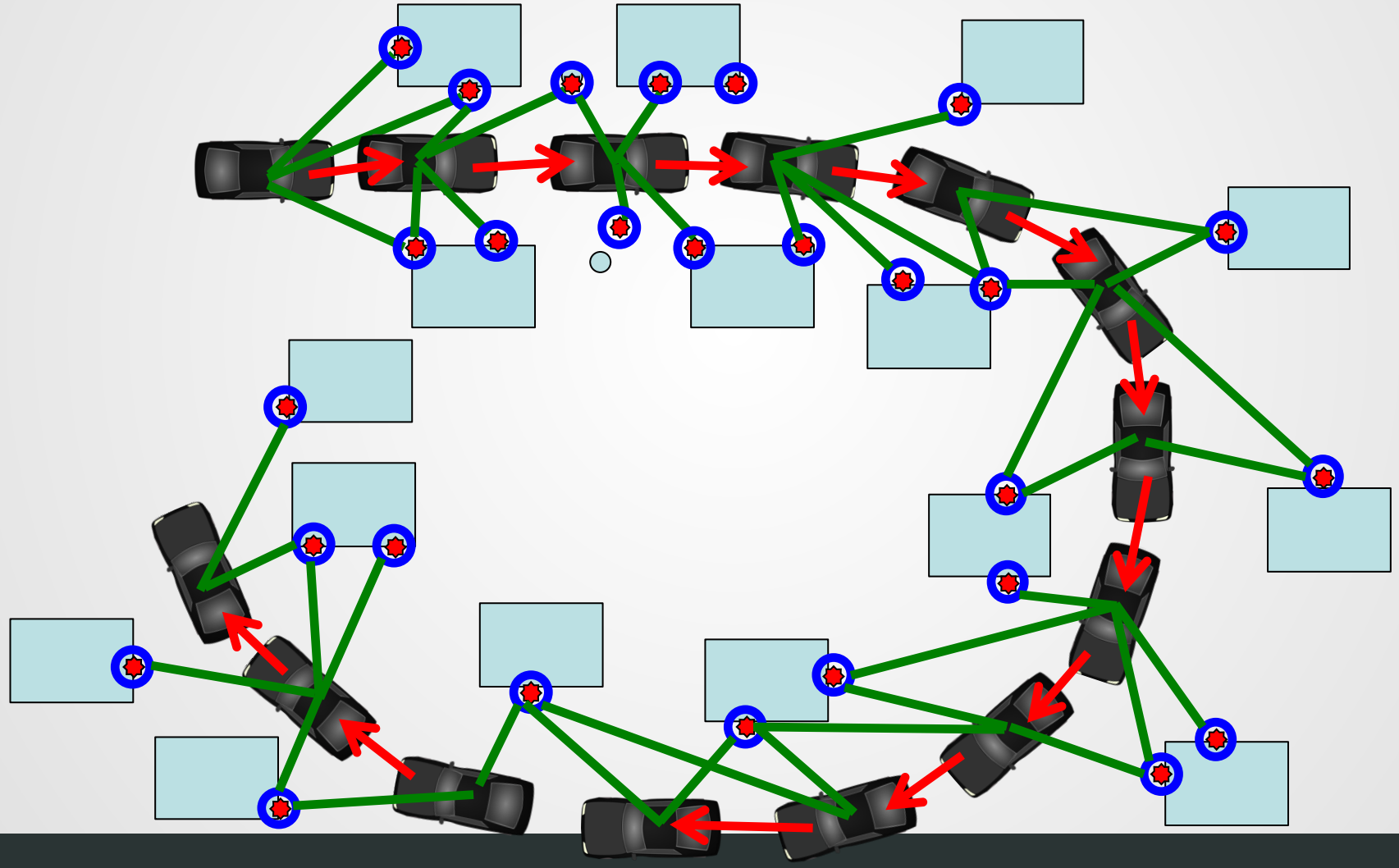
Illustrating Loop Closure



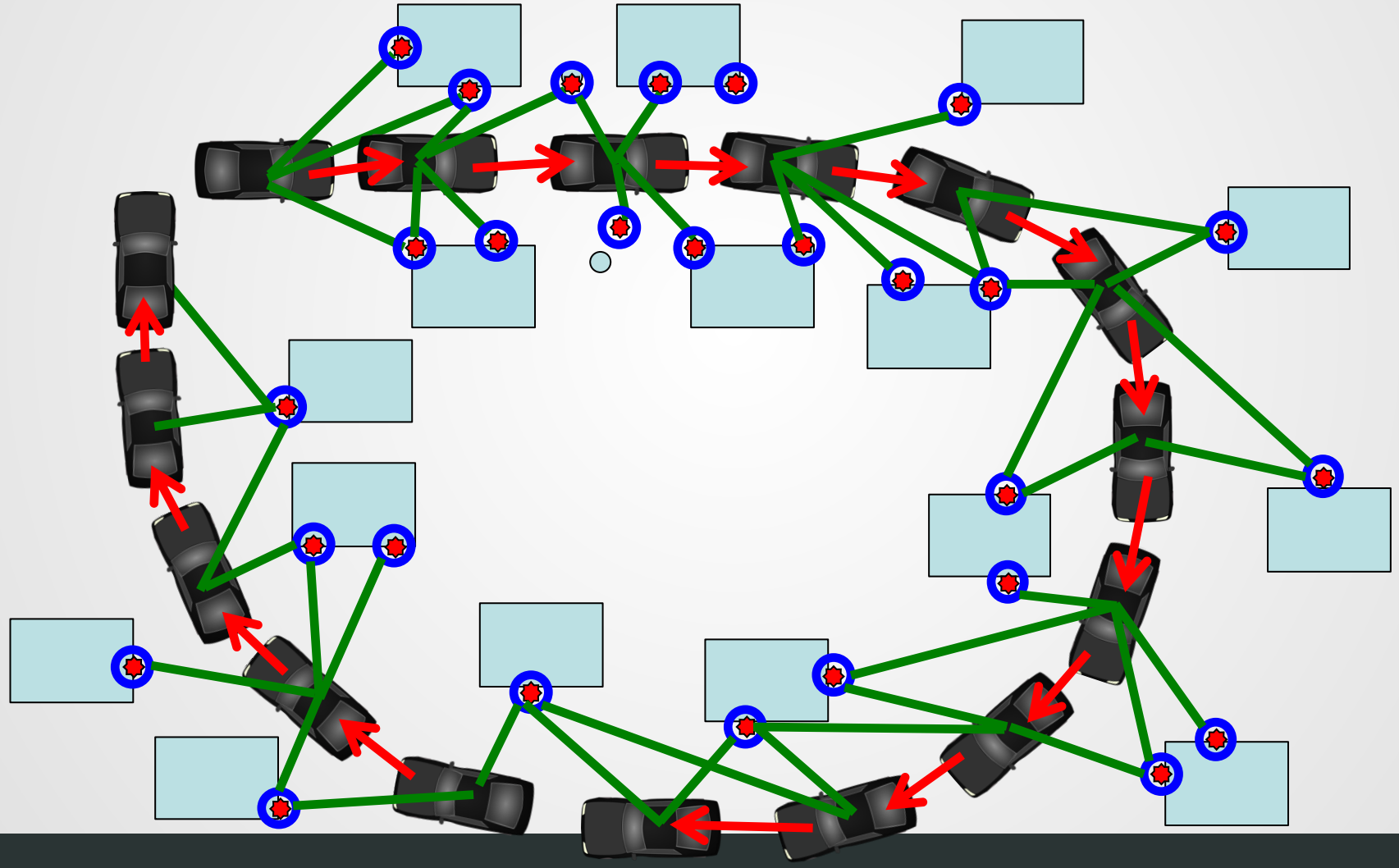
Illustrating Loop Closure



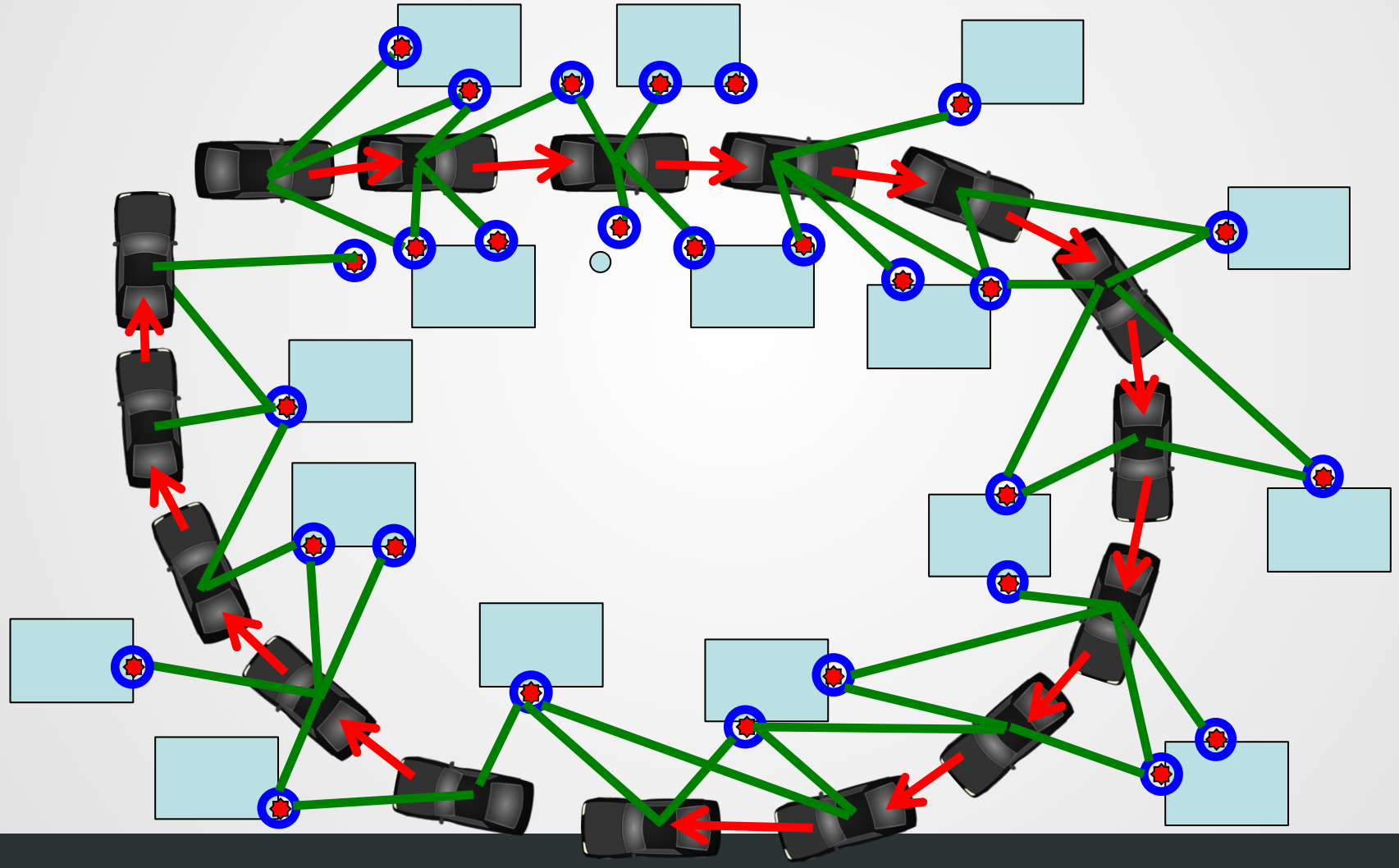
Illustrating Loop Closure



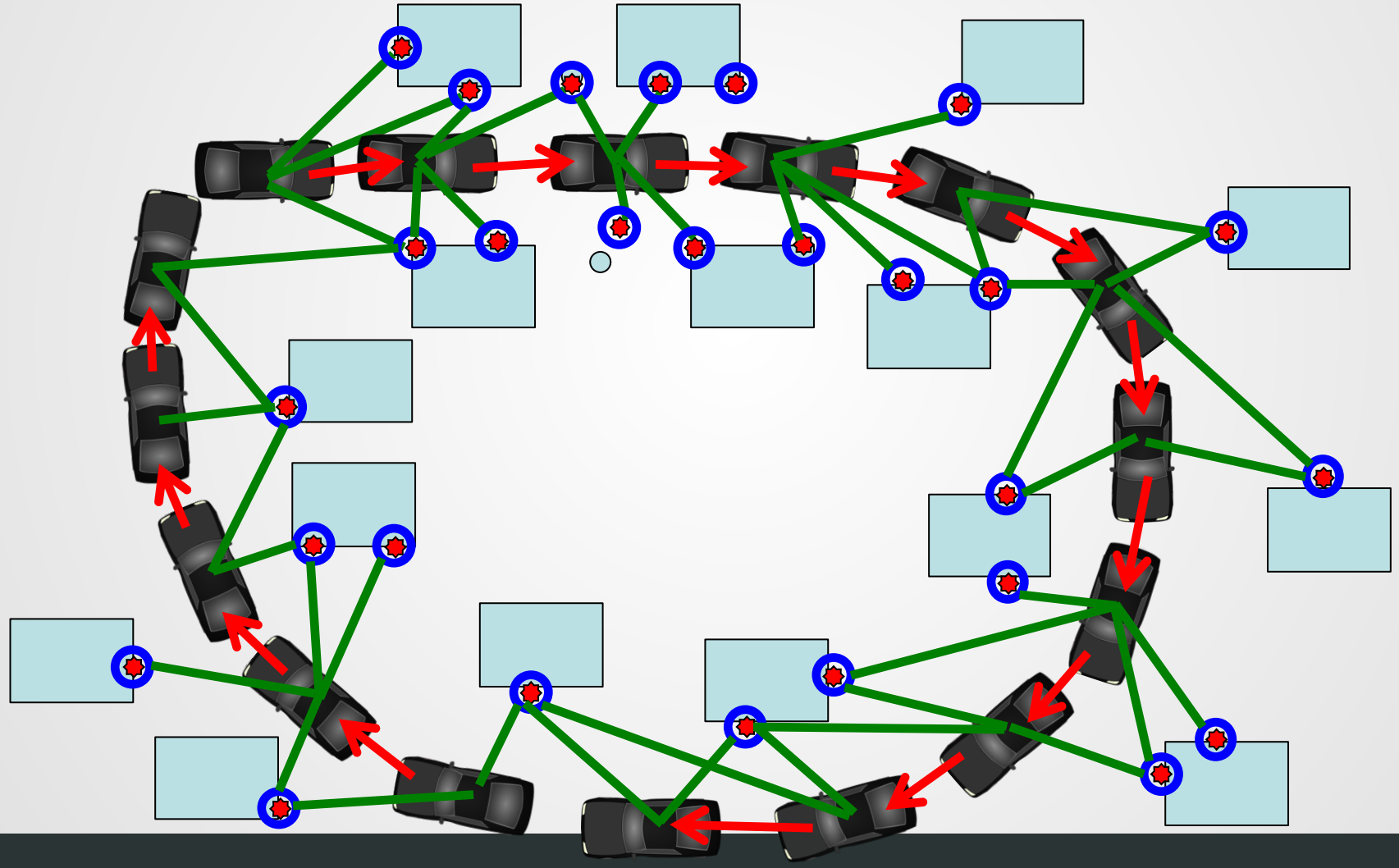
Illustrating Loop Closure



Illustrating Loop Closure



Illustrating Loop Closure



SLAM Examples

- *Formulating SLAM as a Graph Problem*
- **SLAM Examples**
- *Computational and Storage Costs*

Linear SLAM Example

- We'll use the example from Lab 02 in which we have a fully linear system
- The Kalman filter is *guaranteed to work*
- The Kalman filter is *optimal*
- We can see how to implement this and compare the performance directly with the Kalman filter

Platform State and Process Model

- The state of the platform is

$$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \end{bmatrix}$$

Platform Process Model

- **The process model is**

$$\mathbf{x}_{k+1} = \mathbf{F}_{k+1}\mathbf{x}_k + \mathbf{B}_{k+1}\mathbf{u}_{k+1} + \mathbf{v}_{k+1}$$

Error for the Process Model Factor

- The error for the process model factor is given by

$$\mathbf{e}\left[\hat{\mathbf{x}}_{k+1}, \hat{\mathbf{x}}_k, \mathbf{u}_{k+1}\right] = \hat{\mathbf{x}}_{k+1} - \mathbf{F}_{k+1}\hat{\mathbf{x}}_k - \mathbf{B}_{k+1}\mathbf{u}_{k+1}$$

Landmark State

- **The landmark state is its position**

$$\mathbf{m}^i = \begin{bmatrix} x^i \\ y^i \end{bmatrix}$$

Landmark Observation Equation

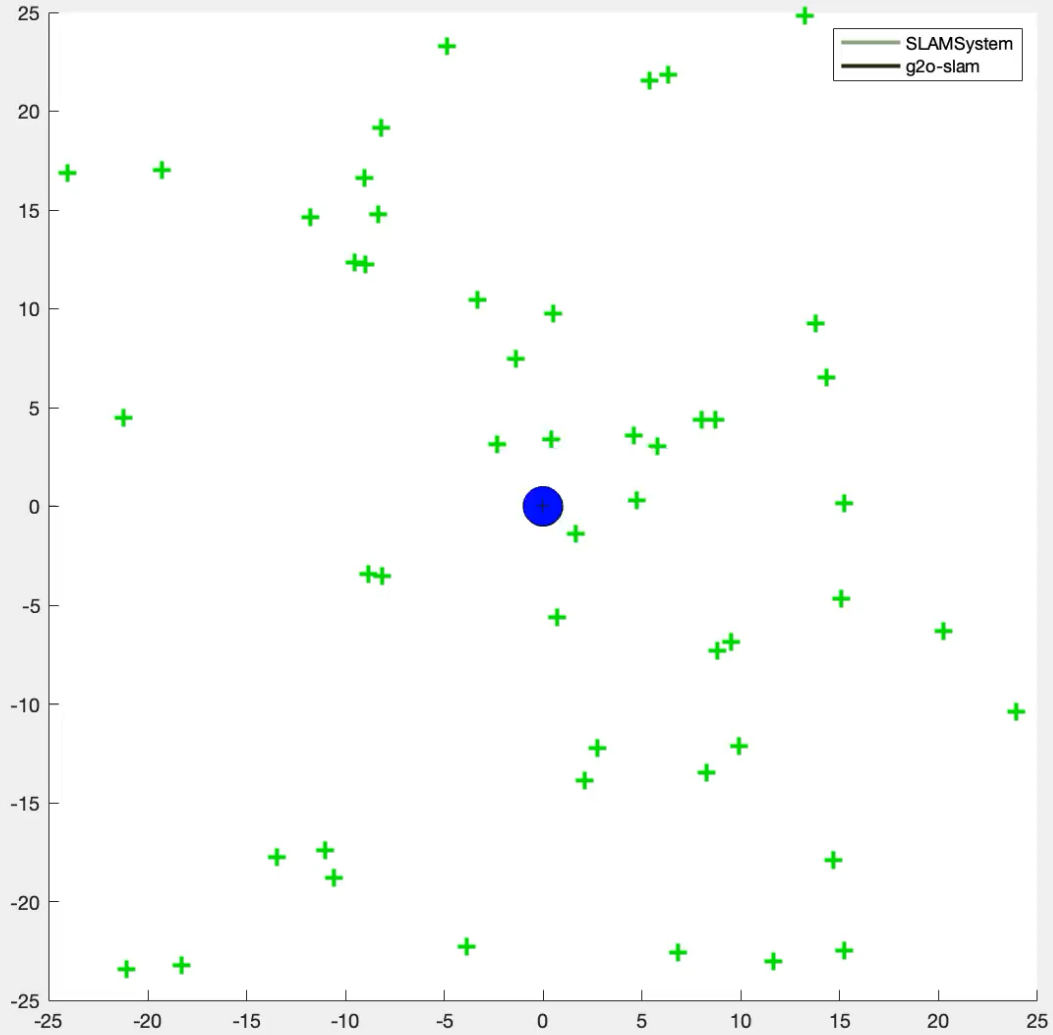
- **The landmark observation equation is**

$$\mathbf{z}_{k+1}^{j} = \mathbf{m}^{i_j} - \mathbf{x}_{k+1} + \mathbf{w}_{k+1}^{j}$$

Landmark Likelihood Factor

- **Therefore, the observation error term is**

$$\mathbf{l}\left[\mathbf{z}_{k+1}^{j}, \hat{\mathbf{m}}^{i_j}, \hat{\mathbf{x}}_{k+1}\right] = \mathbf{z}_{k+1}^{j} - \hat{\mathbf{m}}^{i_j} + \hat{\mathbf{x}}_{k+1}$$


SLAMSystem
g2o-slam
25
20
15
10
5
0
-5
-10
-15
-20
-25
-25
-20
-15
-10
-5
0
5
10
15
20
25

Discussion

- We can see the two *are the same*
- The Kalman filter is optimal and so it's impossible to do better in this case
- Therefore, the fact the graph produces the same results shows that it's optimal as well, but it was put together in a very different way
- Things get interesting when the system is *nonlinear*

Nonlinear SLAM Example

- In this section we'll just show some results for a nonlinear system

- However, since this is central to CW01, most of the details will be omitted

Nonlinear SLAM Example

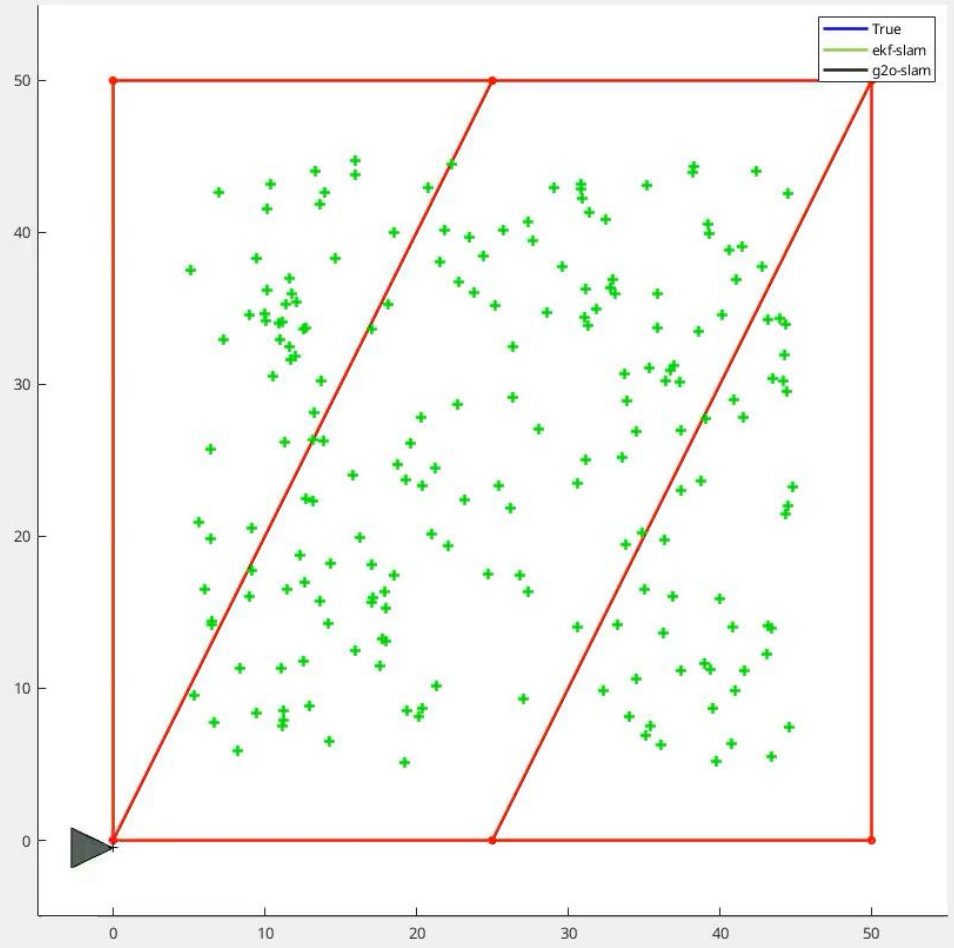
- As before, we have to introduce factors to:
  - Initialize the platform state
  - Predict the platform state over time
  - Handle platform-landmark observations

- In the coursework, you are provided with the code to initialize but you will need to write the rest yourself

Nonlinear SLAM Example

- We will also need to provide initialization code which produces numerical values for
  - To initialize the platform
  - To predict the next state of the platform
  - To initialize the platform estimate

- In the coursework, we provide the code to initialize the platform state, but you'll have to code the other bits yourself


True
ekf-slam
g2o-slam
50
40
30
20
10
0
0
10
20
30
40
50

Computational and Storage Costs

- *Formulating SLAM as a Graph Problem*
- *SLAM Examples*
- Computational and Storage Costs

Storage and Computational Costs in the Kalman Filter

- In a Kalman filter, the storage and computational costs are dominated by the covariance matrix

- This has to store the covariance of the platform, all the landmarks, and all the cross correlations between them

Covariance

$$\mathbf{P}^{s}_{k|k} = \begin{bmatrix} \mathbf{P}^{\mathbf{xx}}_{k|k} & \mathbf{P}^{\mathbf{x}1}_{k|k} & \mathbf{P}^{\mathbf{x}2}_{k|k} & \mathbf{P}^{\mathbf{x}3}_{k|k} & \cdots & \mathbf{P}^{\mathbf{x}N_k}_{k|k} \\ \mathbf{P}^{1\mathbf{x}}_{k|k} & \mathbf{P}^{11}_{k|k} & \mathbf{P}^{12}_{k|k} & \mathbf{P}^{13}_{k|k} & \cdots & \mathbf{P}^{1N_k}_{k|k} \\ \mathbf{P}^{2\mathbf{x}}_{k|k} & \mathbf{P}^{21}_{k|k} & \mathbf{P}^{22}_{k|k} & \mathbf{P}^{23}_{k|k} & \cdots & \mathbf{P}^{2N_k}_{k|k} \\ \mathbf{P}^{3\mathbf{x}}_{k|k} & \mathbf{P}^{31}_{k|k} & \mathbf{P}^{32}_{k|k} & \mathbf{P}^{33}_{k|k} & \cdots & \mathbf{P}^{3N_k}_{k|k} \\ \vdots & \vdots & \ddots & \vdots & & \\ \mathbf{P}^{N_k\mathbf{x}}_{k|k} & \mathbf{P}^{N_k1}_{k|k} & \mathbf{P}^{N_k2}_{k|k} & \mathbf{P}^{N_k3}_{k|k} & \cdots & \mathbf{P}^{N_kN_k}_{k|k} \end{bmatrix}$$

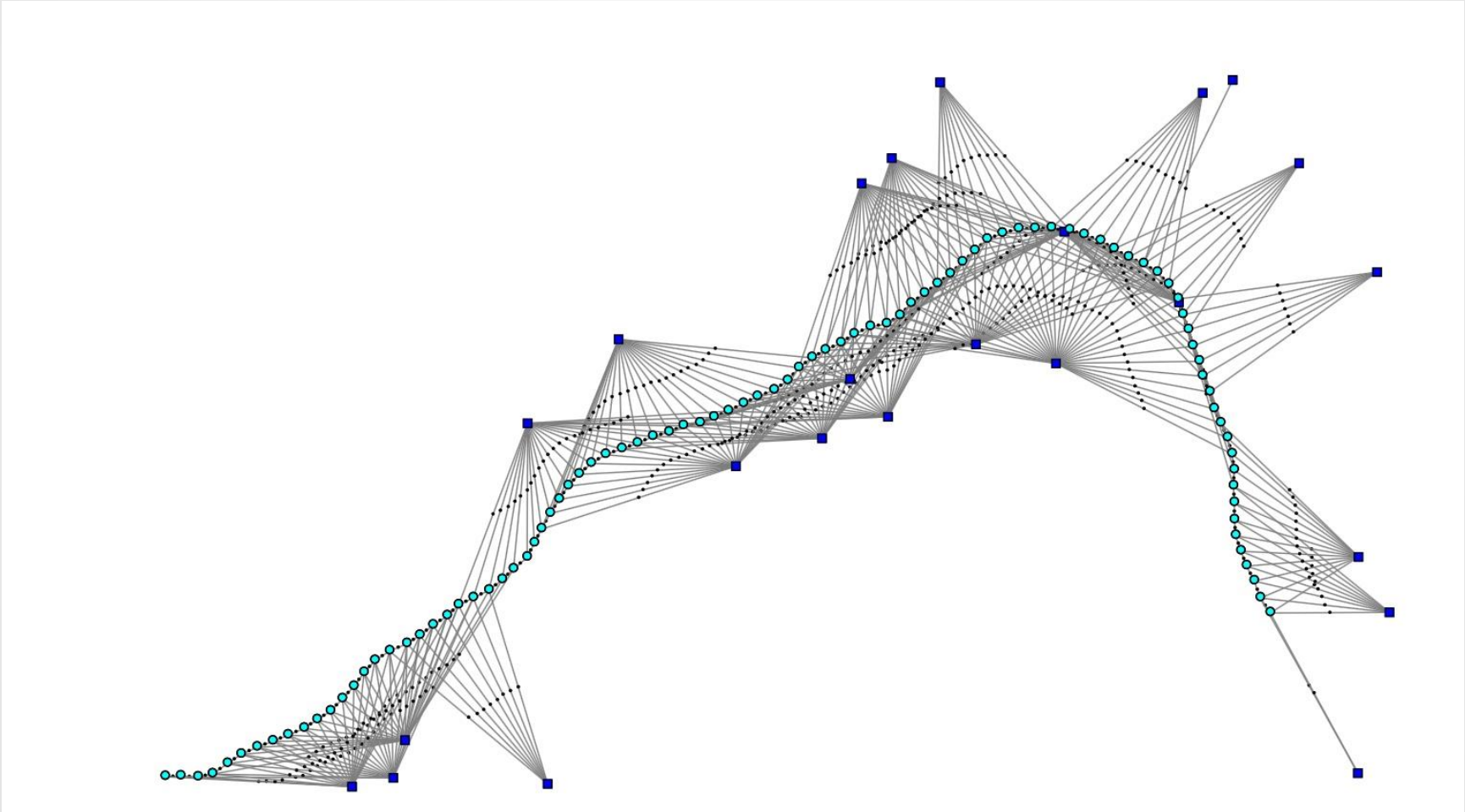
Storage and Computational Costs in the Factor Graph

- In the factor graph there are two main phases of operation:
  - Event recording
  - Graph optimization

Event Recording

- When we receive information we record it
- The way we do it is we instantiate additional vertices and edges
- Allocating each new element is constant time
- Storage grows *linearly* over time

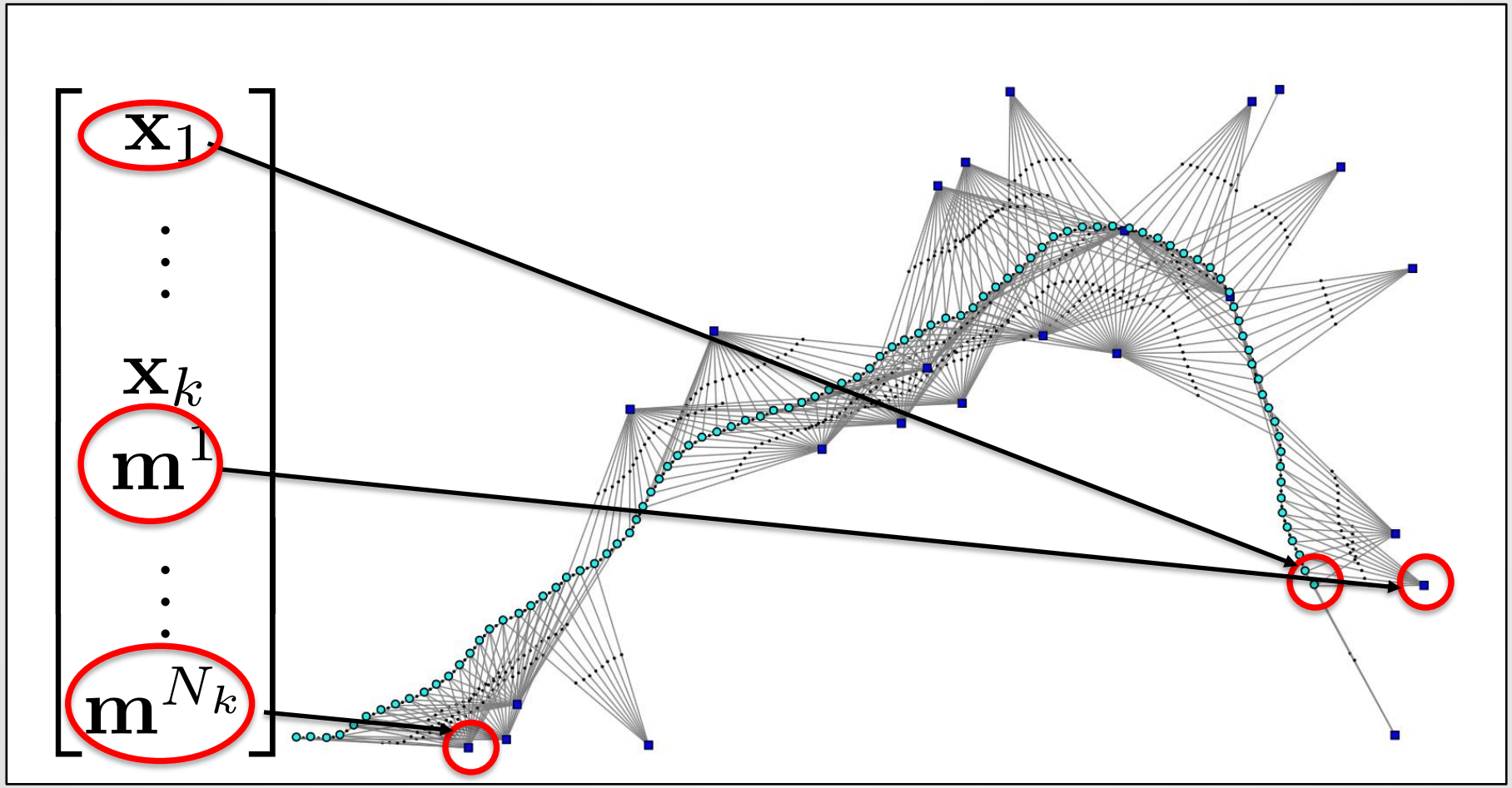
Growing Factor Graph



Graph Optimization

- We compute the MAP estimate of the graph
- To do this, we have to optimize
- Therefore, the optimization costs *dominate*

Assembling the System State



Optimization Step

- **The optimizers are iterative,**

$$\mathbf{X}_k^{i+1} = \mathbf{X}_k^i - \boldsymbol{\delta}\mathbf{X}_k^i$$

- **You can show (see the g2o paper) that the optimization step has the form**

$$\boldsymbol{\delta}\mathbf{X}_k^i = \mathbf{H}^{-1}\mathbf{b}$$

Hessian Matrix

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 c}{\partial x_1^2} & \frac{\partial^2 c}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 c}{\partial x_1 \partial x_s} \\ \frac{\partial^2 c}{\partial x_2 \partial x_1} & \frac{\partial^2 c}{\partial x_2^2} & \cdots & \frac{\partial^2 c}{\partial x_2 \partial x_s} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 c}{\partial x_s \partial x_1} & \frac{\partial^2 c}{\partial x_s \partial x_2} & \cdots & \frac{\partial^2 c}{\partial x_s^2} \end{bmatrix}$$

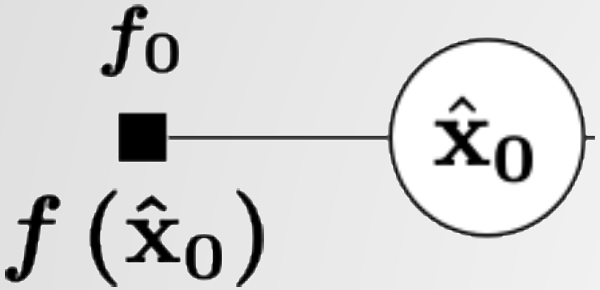
Using the Hessian

- It turns out that the inverse of the Hessian has a very easy form to write down
- Therefore, we compute the inverse of the Hessian directly
- However, the computational and storage costs depend on the number of entries in the Hessian
- Because Hessian encodes the structure of the graph, it is *sparse* – most of the terms are non-zero

Structure of the Hessian

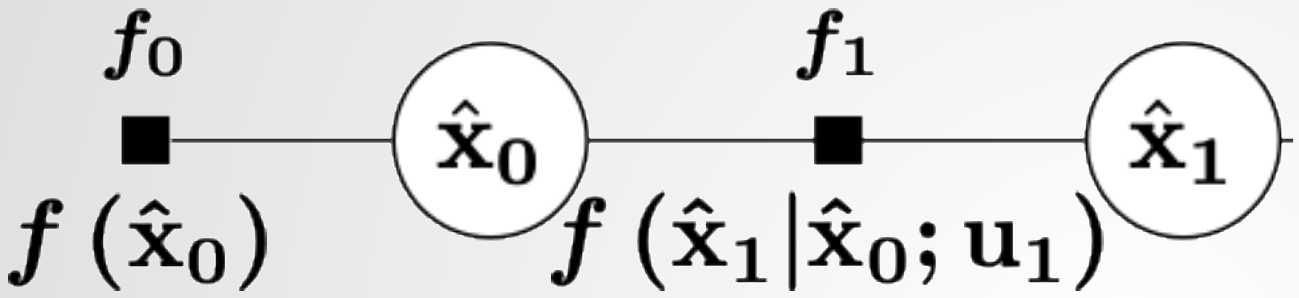
Row \ Col	$\hat{\mathbf{x}}_0$	$\cdots$	$\cdots$	$\cdots$
$\hat{\mathbf{x}}_0$				
$\vdots$				
$\vdots$				
$\vdots$				

Structure of the Hessian for Prediction Problems



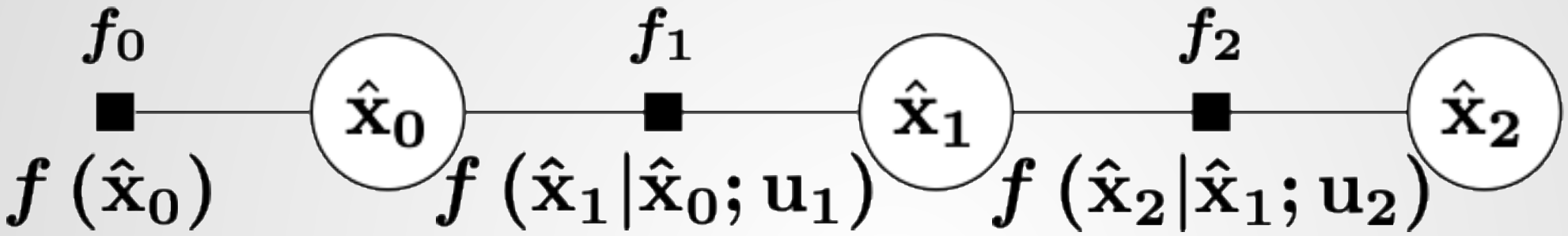
Row \ Col	$\hat{\mathbf{x}}_0$
$\hat{\mathbf{x}}_0$	

Structure of the Hessian for Prediction Problems



Row \ Col	$\hat{\mathbf{x}}_0$	$\hat{\mathbf{x}}_1$
$\hat{\mathbf{x}}_0$		
$\hat{\mathbf{x}}_1$		

Structure of the Hessian for Prediction Problems

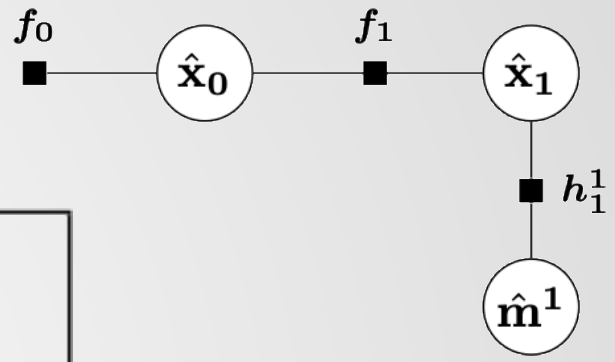


Row \ Col	$\hat{\mathbf{x}}_0$	$\hat{\mathbf{x}}_1$	$\hat{\mathbf{x}}_2$
$\hat{\mathbf{x}}_0$			
$\hat{\mathbf{x}}_1$			
$\hat{\mathbf{x}}_2$			

Structure of the Hessian for Prediction Problems

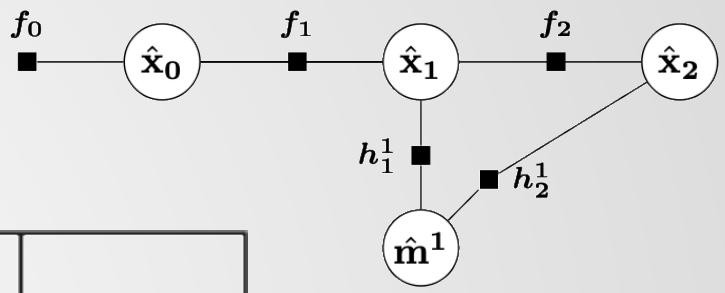
Row \ Col	$\hat{\mathbf{x}}_0$	$\hat{\mathbf{x}}_1$	$\hat{\mathbf{x}}_2$	$\hat{\mathbf{x}}_3$	$\hat{\mathbf{x}}_4$	$\hat{\mathbf{x}}_5$
$\hat{\mathbf{x}}_0$						
$\hat{\mathbf{x}}_1$						
$\hat{\mathbf{x}}_2$						
$\hat{\mathbf{x}}_3$						
$\hat{\mathbf{x}}_4$						
$\hat{\mathbf{x}}_5$						

Structure of the Hessian with Landmarks

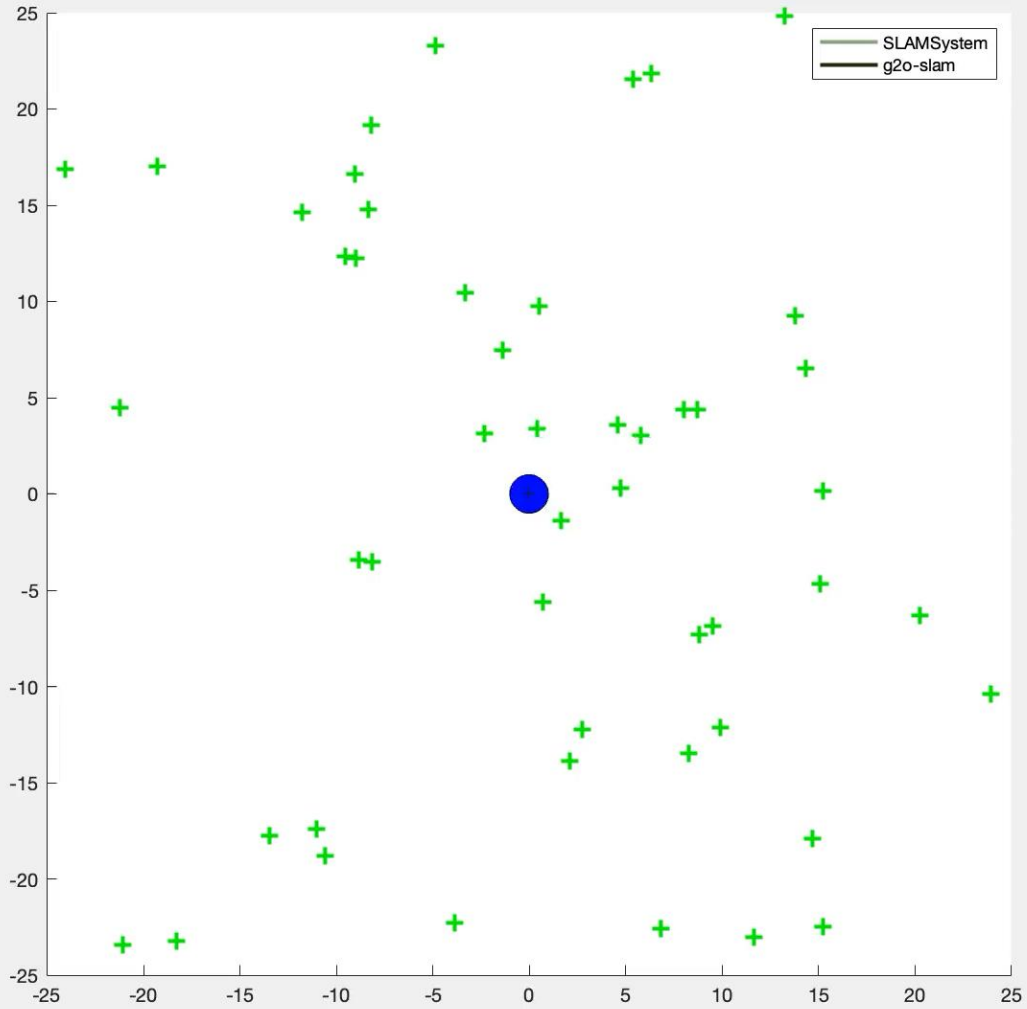


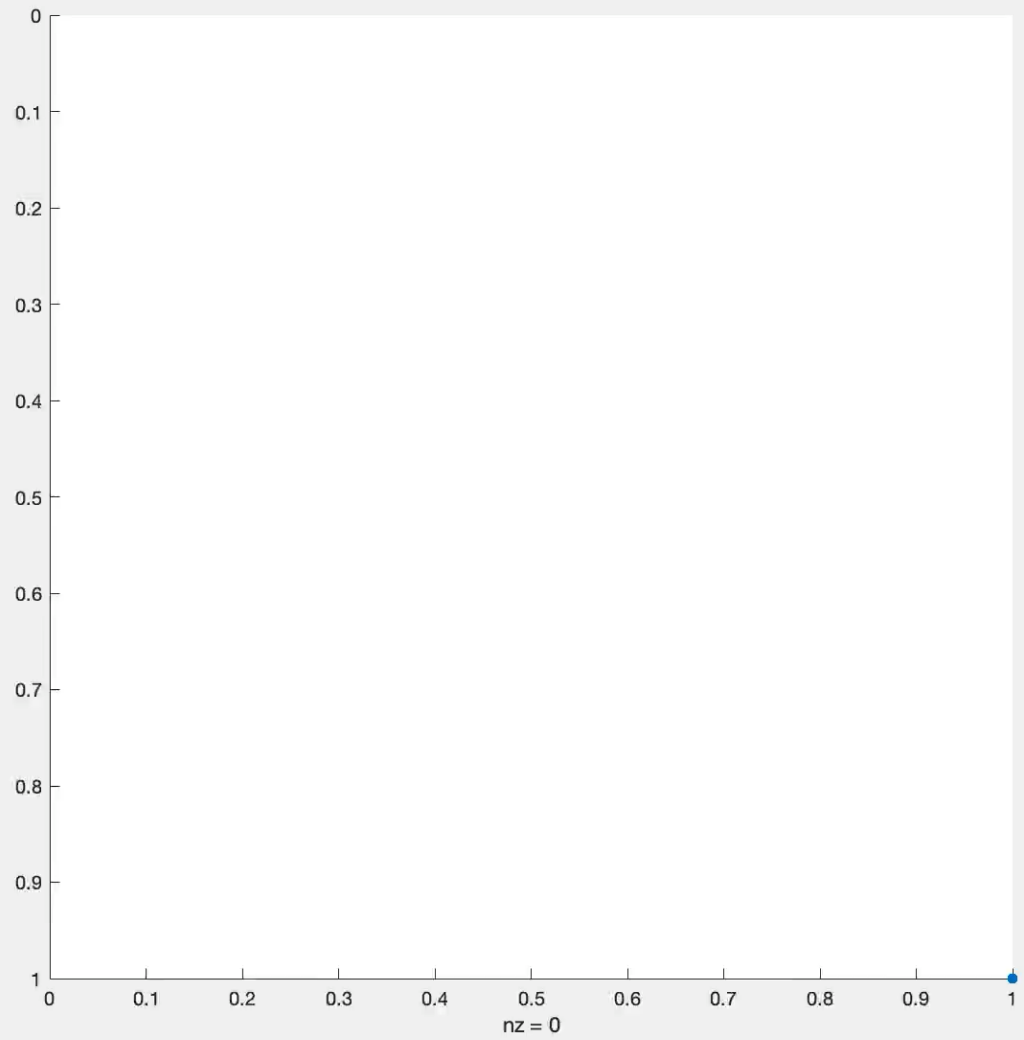
Row \ Col	$\hat{\mathbf{x}}_0$	$\hat{\mathbf{x}}_1$	$\hat{\mathbf{m}}^1$
$\hat{\mathbf{x}}_0$			
$\hat{\mathbf{x}}_1$			
$\hat{\mathbf{m}}^1$			

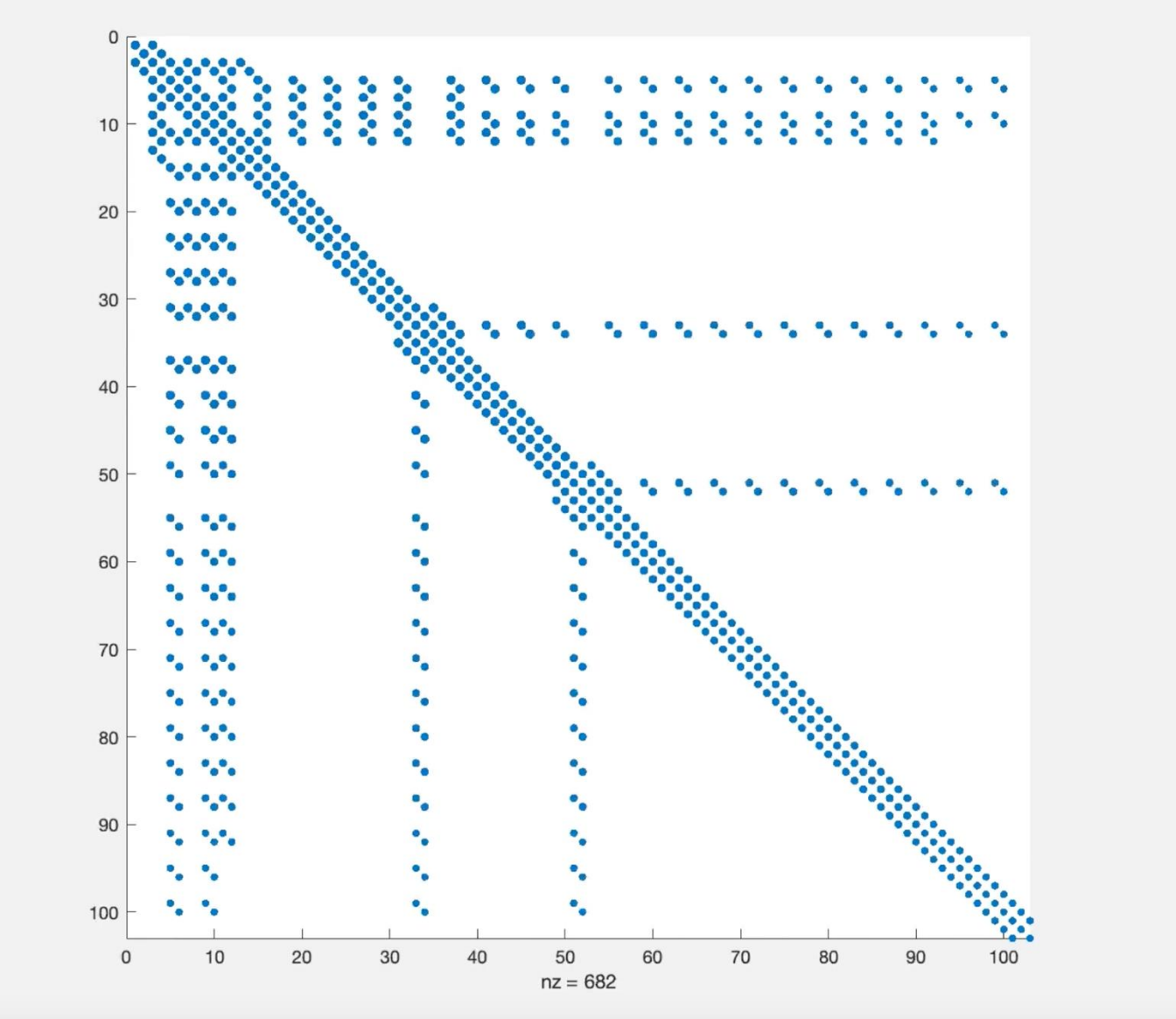
Structure of the Hessian with Landmarks



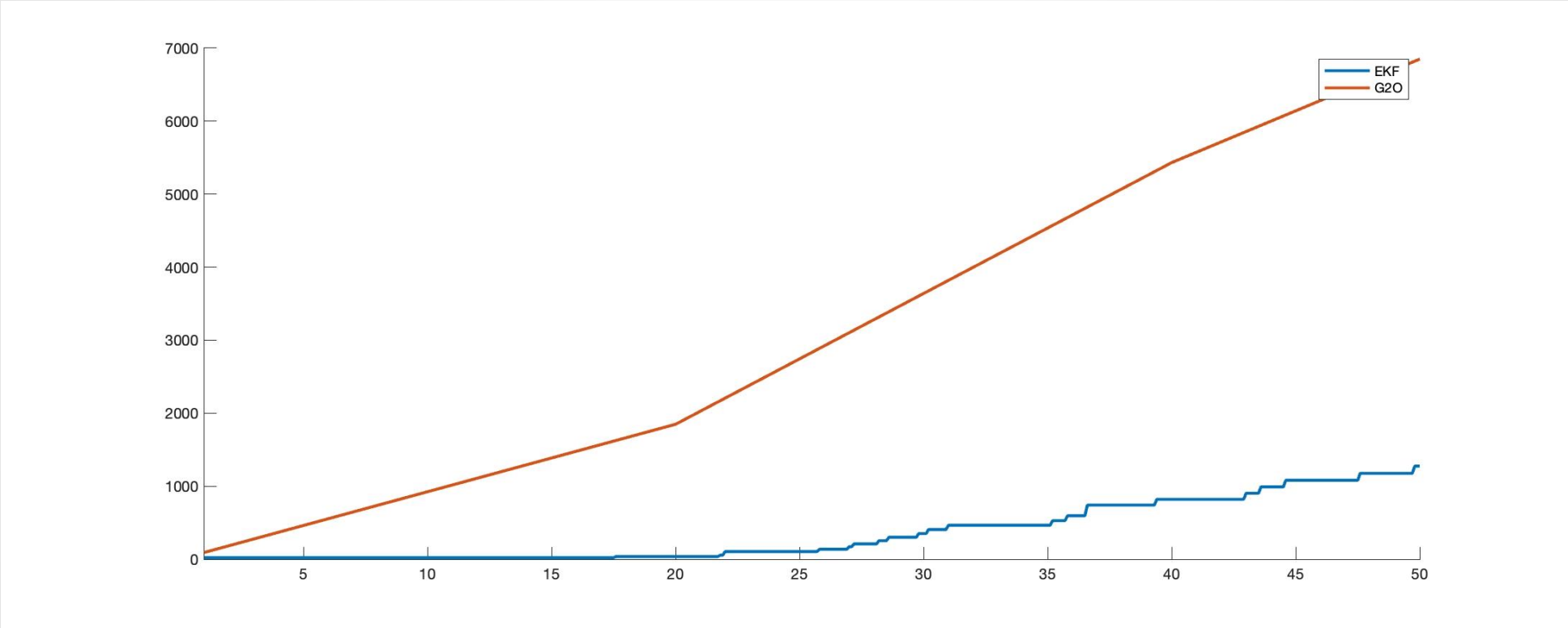
Row \ Col	$\hat{\mathbf{x}}_0$	$\hat{\mathbf{x}}_1$	$\hat{\mathbf{m}}^1$	$\hat{\mathbf{x}}_2$
$\hat{\mathbf{x}}_0$				
$\hat{\mathbf{x}}_1$				
$\hat{\mathbf{m}}^1$				
$\hat{\mathbf{x}}_2$				

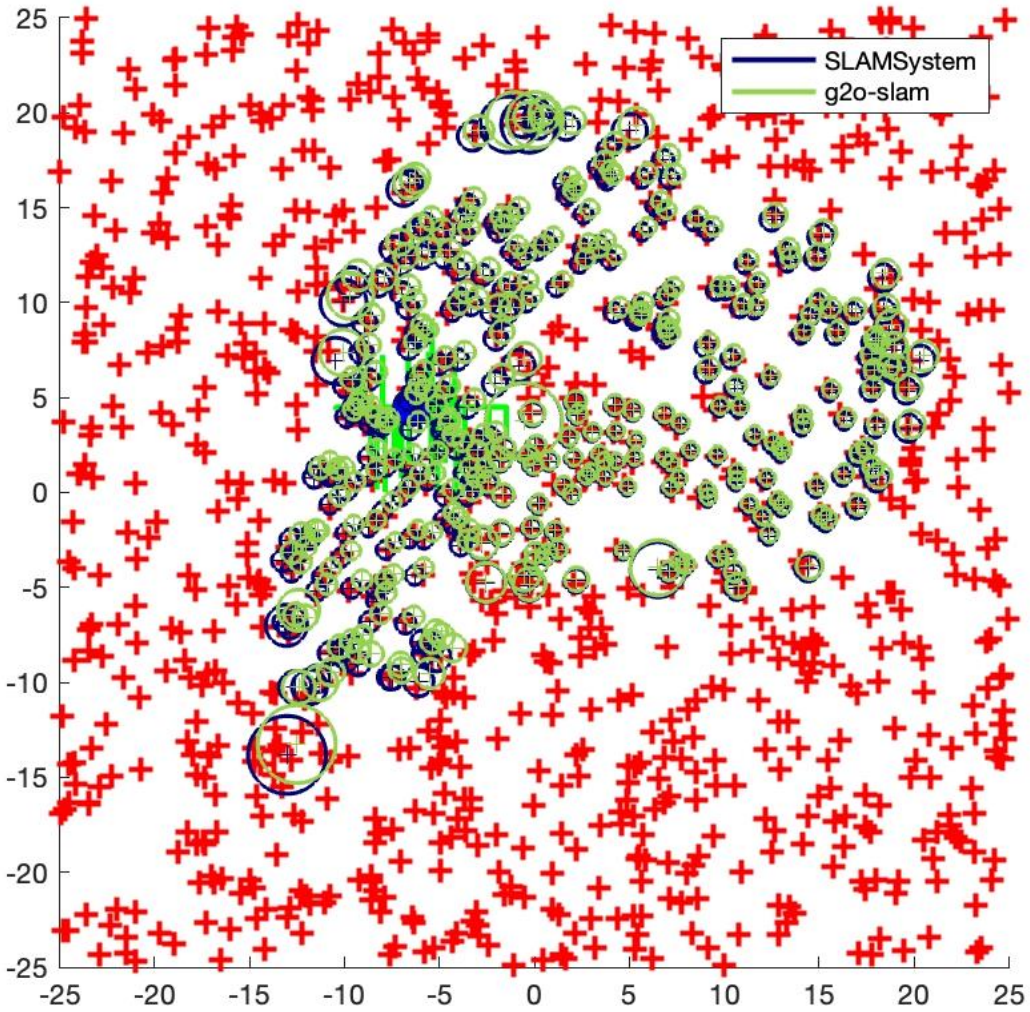

SLAMSystem
g2o-slam
25
20
15
10
5
0
-5
-10
-15
-20
-25
-25
-20
-15
-10
-5
0
5
10
15
20
25

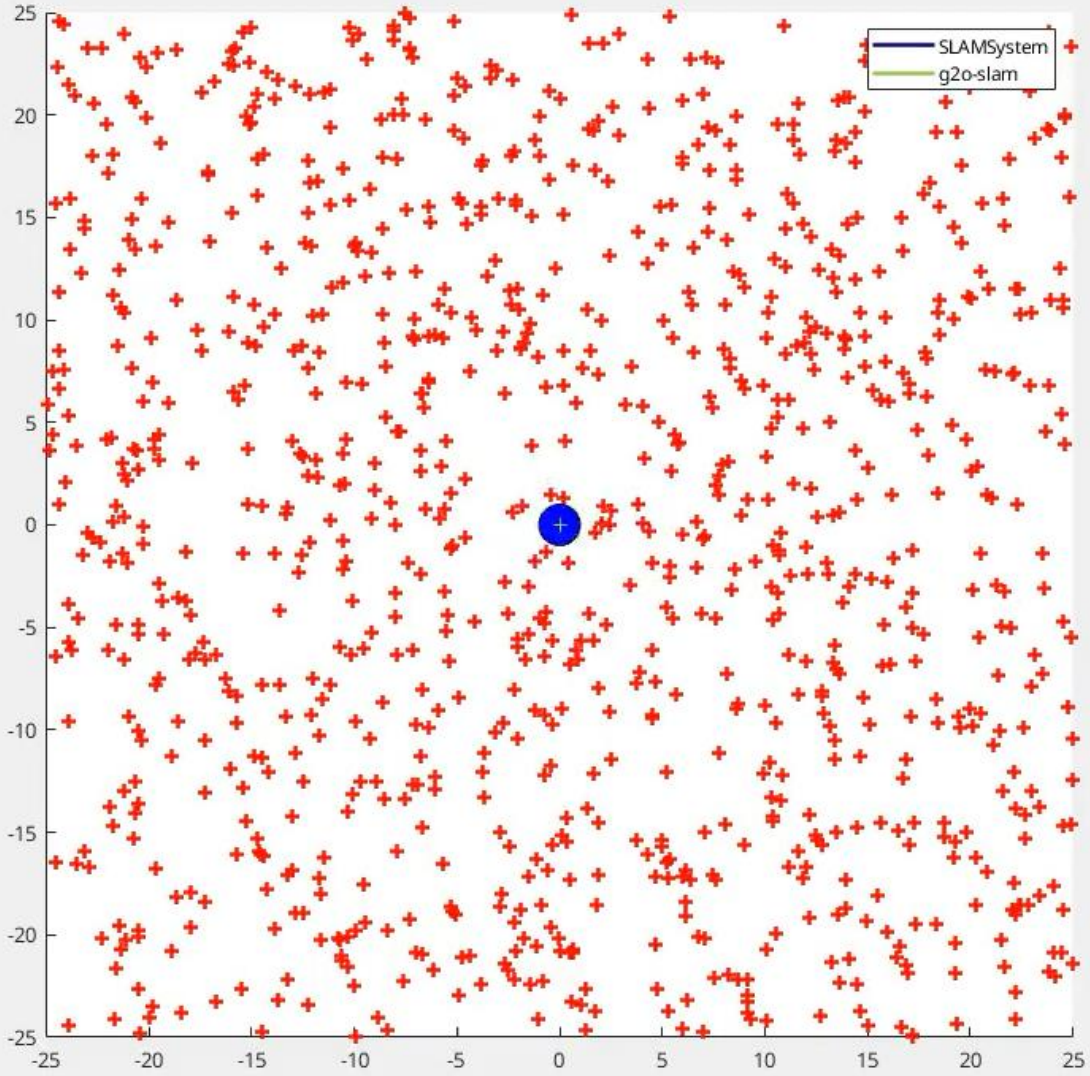

0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
nz = 0

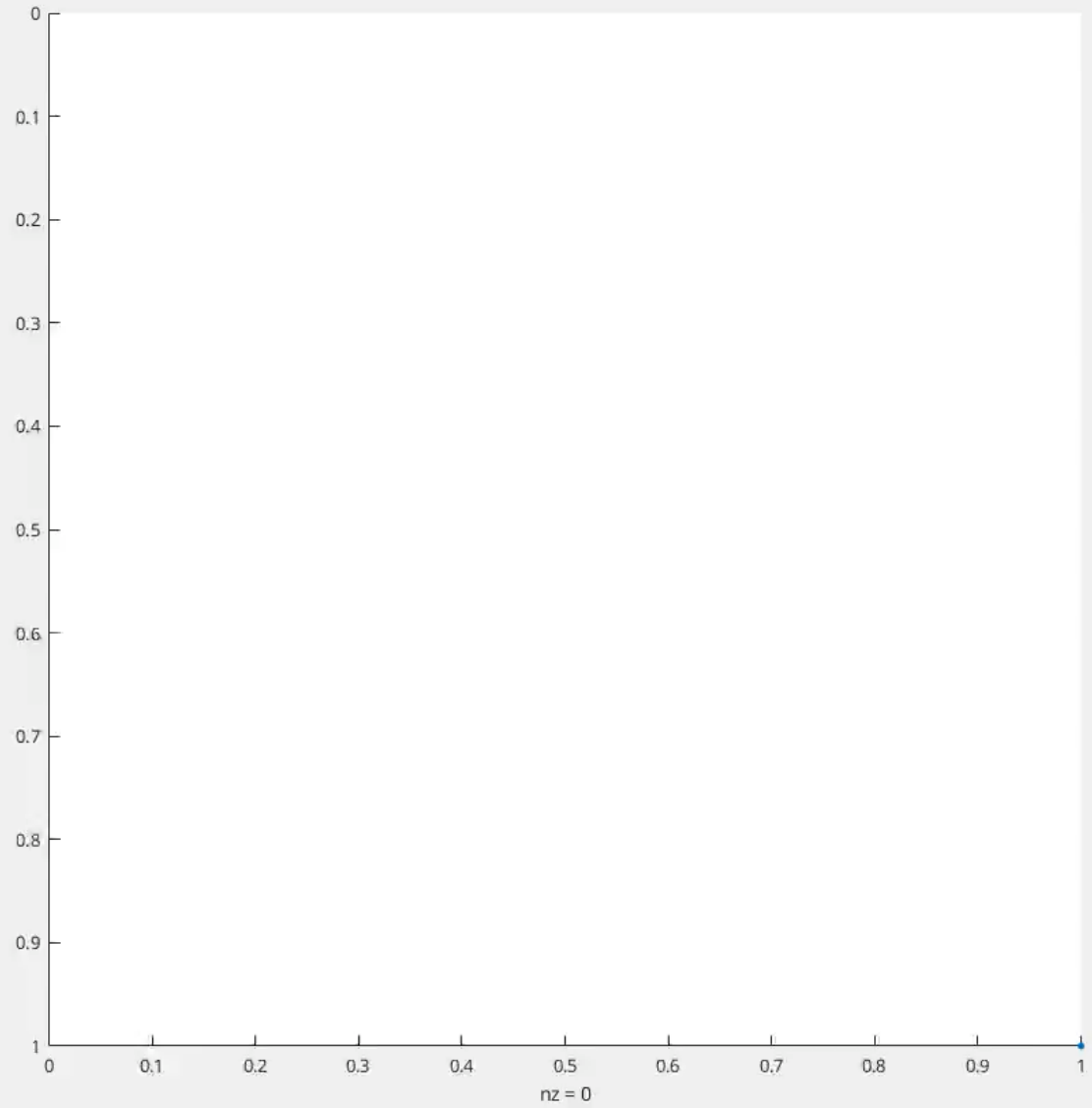

0
10
20
30
40
50
60
70
80
90
100
0
10
20
30
40
50
60
70
80
90
100
nz = 682

Small Scenario Storage

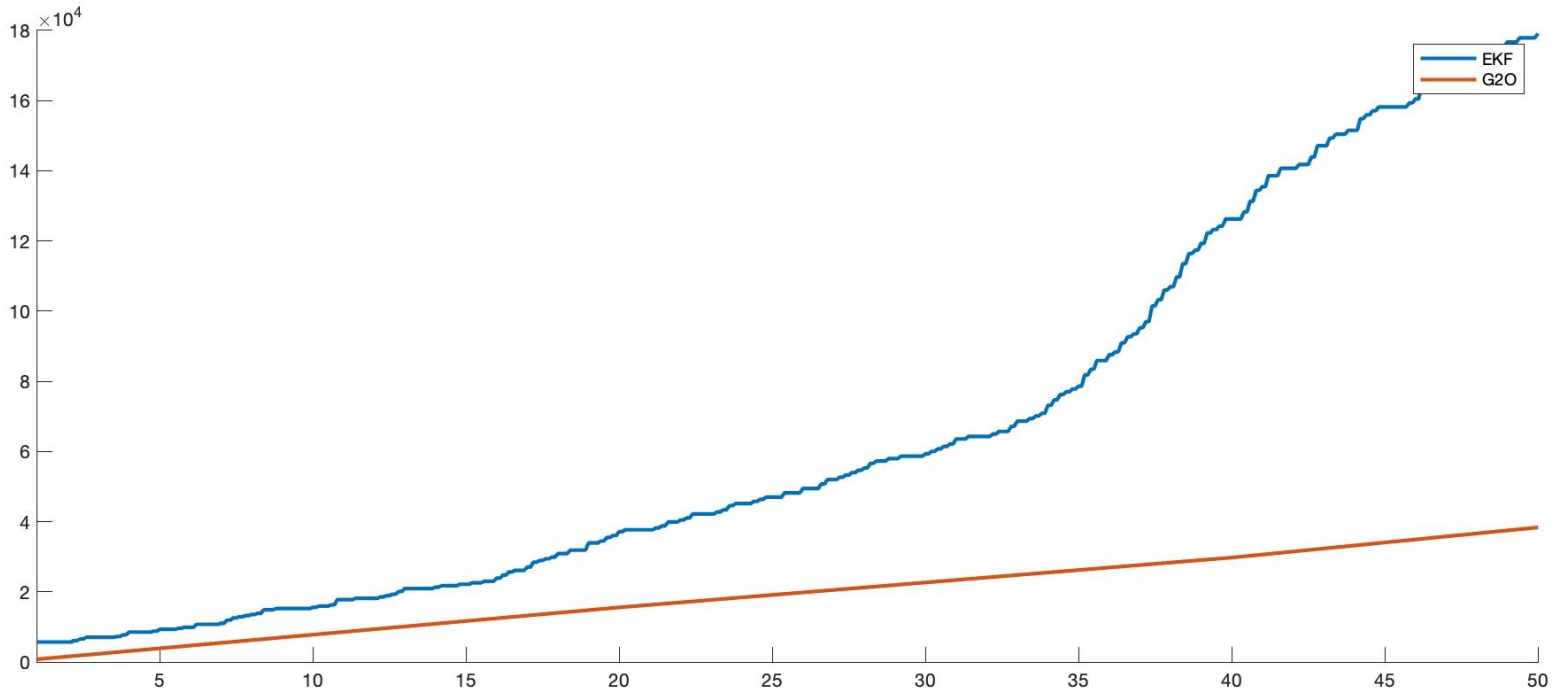



SLAMSystem
g2o-slam
25
20
15
10
5
0
-5
-10
-15
-20
-25
-25
-20
-15
-10
-5
0
5
10
15
20
25


SLAMSystem
g2o-slam
25
20
15
10
5
0
-5
-10
-15
-20
-25
-25
-20
-15
-10
-5
0
5
10
15
20
25


0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
nz = 0

Large Scenario Storage



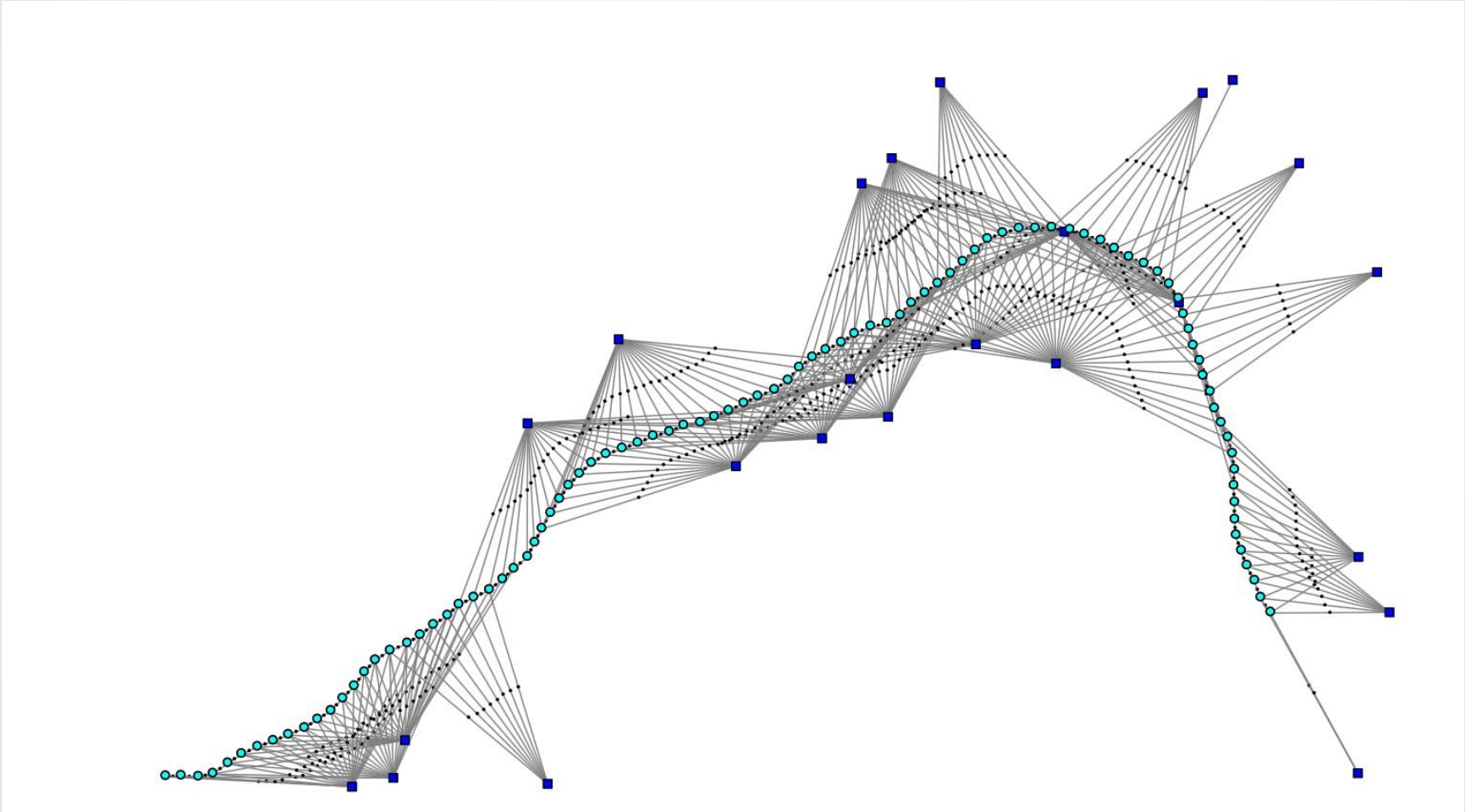
Improving Computation Further

- There are a lot of strategies we can use:
  - Selective optimization
  - Graph pruning
  - Controlling how landmarks enter the map

Selective Optimization

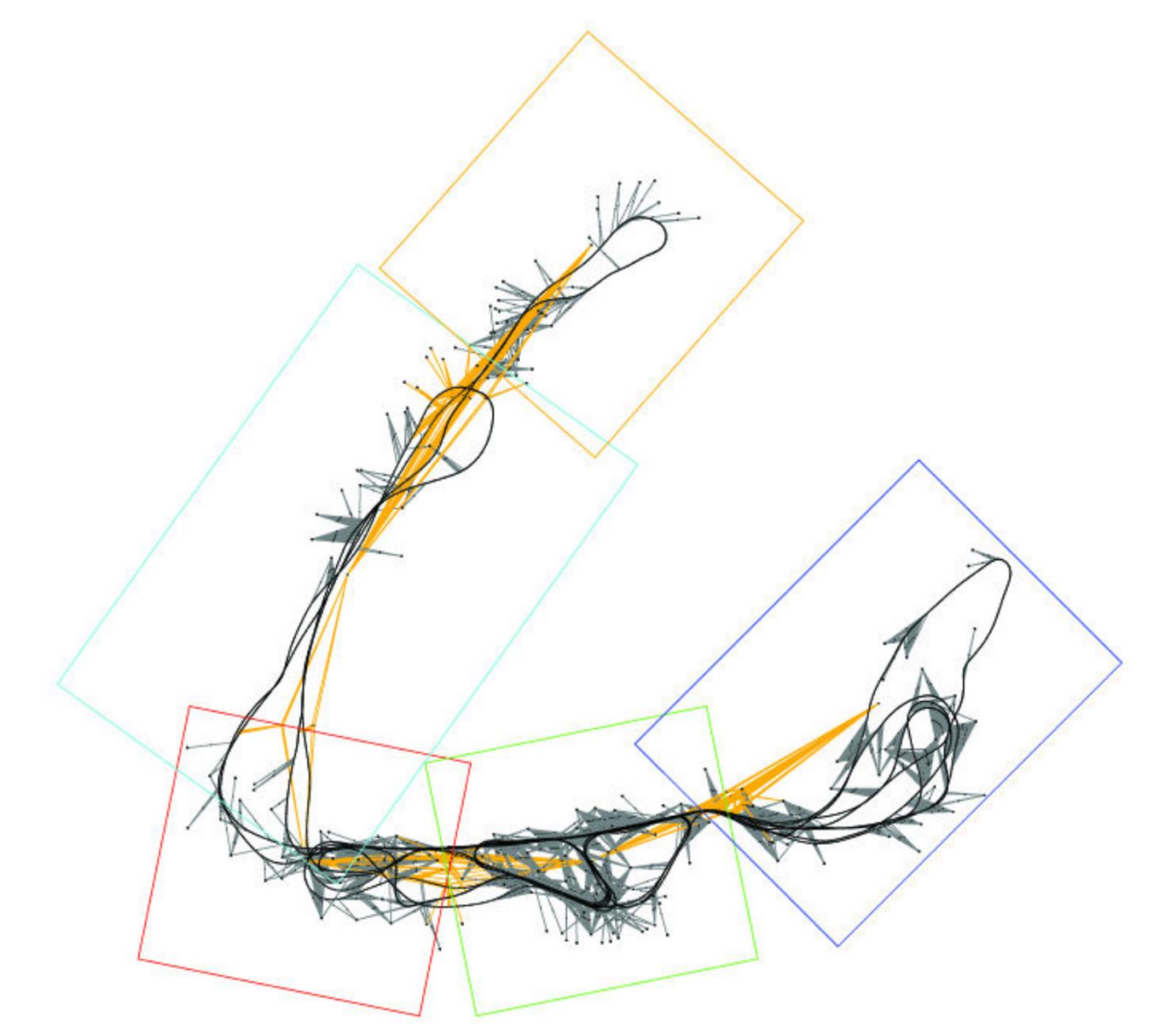
- You don’t have to optimize the whole graph at every time step
- You can choose to update just a subset of the vertices, which greatly reduces the costs

Fixing Vertices



Selective Optimization

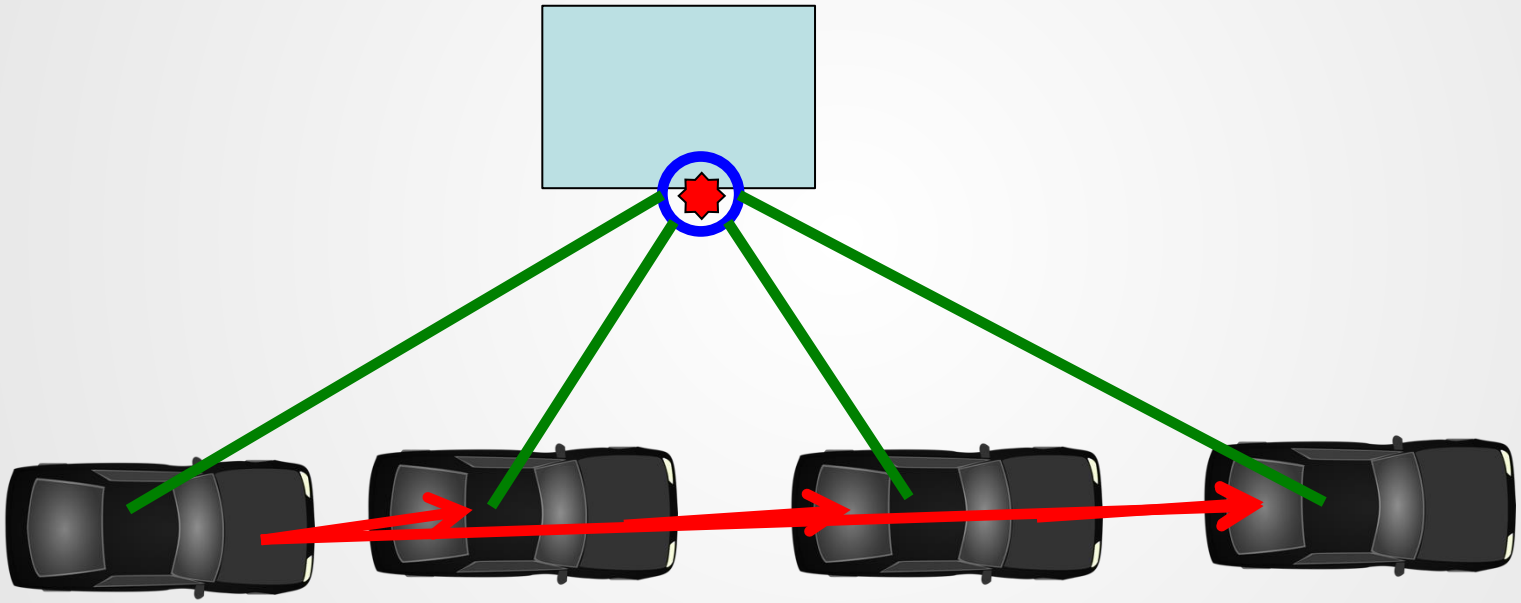
- You don’t have to optimize the whole graph
- Rather, you can choose to update just a subset of the vertices, which greatly reduces the costs
- There are various schemes, but mathematically they are the same as *fixing* a vertex
- When a vertex is fixed, its value is held constant and you can avoid ever putting it into the optimization step



Graph Pruning

- Any landmark will be observed multiple times by a vehicle
- We've already seen that the covariance on the landmarks falls more slowly the more you observe them
- Therefore, you can end up with a lot of observations (=platform states, edges) which contribute little to improving the overall accuracy

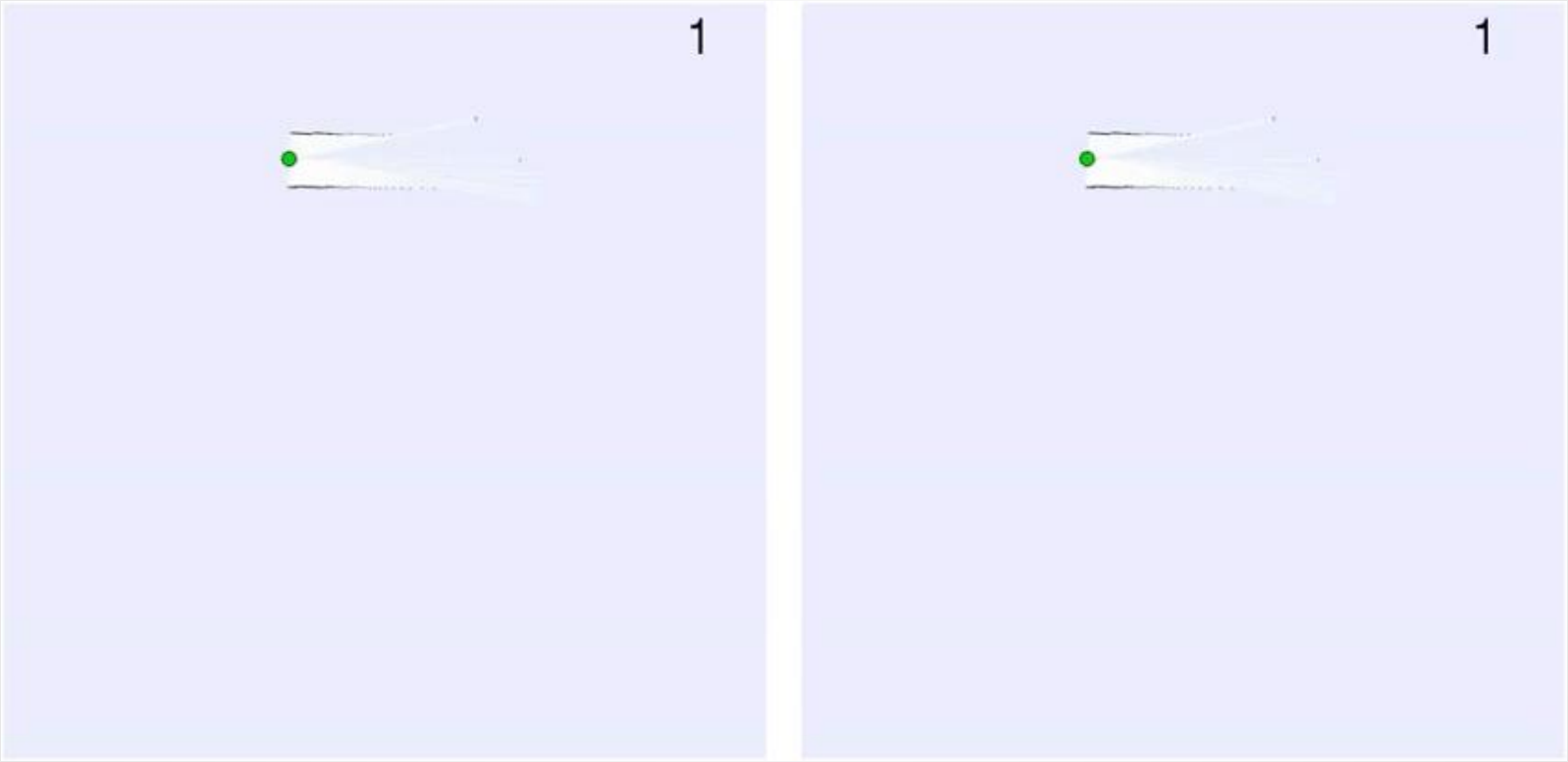
Graph Pruning



Graph Pruning through Marginalization

- One way to delete a vertex or an edge is to simply remove it from the graph
- However, this causes a loss of information
- Therefore, a lot of approaches marginalize out vertices instead, often using various Gaussian assumptions

Pruning Vertices from the Map

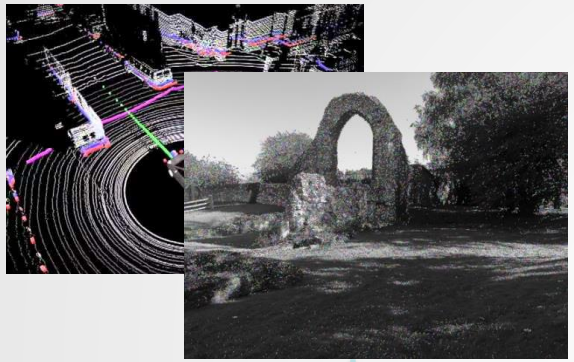


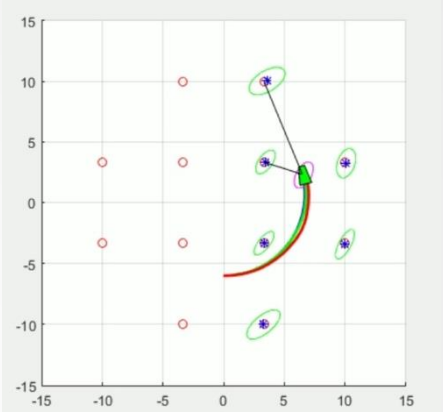
From “SLAM - Graph-based SLAM with Node Reduction”

Restricting What Goes into the Map

- Another approach is to restrict what get puts into a map in the first place
- A very common approach is to use a gateway and have an idea of key-frame based SLAM
- A frontend and a back end are used

Two End Design Limits What Goes In





Frontend

Feature Map

Backend

Subset of "interesting" information

Summary

- The application of factor graphs to SLAM is very easy:
  - You create new vertices for each new landmark
  - You connect edges whenever a landmark is observed
- These store the full dependency structure, but in a much simpler way (for the end user!)
- This is all very theoretical though
- Let's start seeing how to build vision-based SLAM systems in practice